
VELAFLOW

COMPLETE TECHNICAL REFERENCE

Self-Hosted AI Productivity Automation System

21 April 2026 - v1.0

Root README.md

VelaFlow

An AI coordination platform that turns a user's fragmented personal tools — tasks, calendar, email, notes, documents — into a single, tenant-owned automated workflow. Built privacy-first: VelaFlow operators never see a tenant's third-party credentials, and an attacker with shell access inside the host cannot read them either.

Table of Contents

1. Vision
 2. Why VelaFlow exists — problem, solution, consequence
 3. Familiar-stack translation
 4. Product Overview
 5. Subscription Tiers
 6. Security and Privacy Model
 7. Engineering Status
 8. Architecture (Summary)
 9. API Surface
 10. Deployment
 11. Roadmap
 12. Repository Layout
 13. Authorship and provenance
 14. License
-

1. Vision

Modern productivity tooling is fragmented. A single user typically operates across a task manager, a calendar, an email account, a notes system, and a growing list of AI assistants. The resulting context-switching tax is large, prioritisation is lossy, and personal data flows through several third-party vendors the user did not choose to combine.

VelaFlow is a single-tenant-per-customer AI coordination layer that sits between those tools. A tenant connects the accounts they already use, selects a subscription tier, and receives a personalised daily briefing, scheduled automations, and — on paid tiers — private LLM inference and retrieval against their own documents. No tenant credential is ever readable by the operator, and no tenant can read another tenant's data.

The commercial proposition is straightforward:

- **Free tier** proves the product with a fixed daily briefing and a small task ceiling.
- **Standard, Premium, and VIP tiers** unlock customisable schedules, delivery channels, retrieval over personal documents, and private LLM inference with optional GPU.
- The platform is engineered to host a single operator, a family, or a small cohort of paying tenants on a commodity VPS, and to scale horizontally to thousands of tenants under Kubernetes with KEDA autoscaling.

VelaFlow is deploy-ready today for an operator running a handful of users. The v1.2 milestone delivers a fully graphical per-tenant workflow editor; until then personalisation is driven through a tier-gated self-service GUI and a stable REST API.

2. Why VelaFlow exists — problem, solution, consequence

This section is written for readers who prefer specifications over narrative. Numbers, not adjectives.

The tenant-facing problem (measurable, not rhetorical)

- A single knowledge worker operates across **6–8 disconnected SaaS surfaces** (task manager, calendar, email, notes, document store, chat, plus one or more AI assistants). Each surface is a separate auth domain, a separate data silo, and a separate notification stream.
- Context reconstruction after an interruption costs **15–25 minutes** of re-orientation before productive work resumes; this cost compounds with every surface added.
- Prioritisation is **lossy by construction**: each tool ranks its own items against its own signals, with no cross-surface view. Urgent email, stale tasks, and calendar conflicts are never ranked on the same axis.
- The surface count equals the **breach surface**: every vendor added multiplies the probability that one of them leaks credentials, profile data, or message content. No tenant chose that product combination as a deliberate security decision — it accreted.

The operator-facing problem (where most “AI productivity”

stacks fail)

- Most multi-tool assistants require the operator to hold the tenant’s third-party credentials **in cleartext or under a single shared key**. A single root shell, a single leaked backup, or a single disgruntled operator retrieves every tenant’s tokens at once. This is the LastPass-2022 and the Okta-2023 failure mode, applied to productivity data.
- Most “AI-first” platforms make the LLM a hard runtime dependency. When the upstream provider rate-limits, outages, deprecates a model, or changes pricing overnight, the product becomes unusable. There is no deterministic fallback path.
- Multi-tenant SaaS frequently relies on **row-level filters in shared tables** without a cryptographic boundary. A malformed query, an ORM bug, or a misfiled **WHERE** clause leaks cross-tenant data in a way no pentest catches until it is in production.

The solution — exactly what VelaFlow does

- **One API factory** (`brain.api.app:create_app`) serves every endpoint on `:8765`; every request passes `HTTP5OnlyMiddleware` as the outermost layer (`HTTP` → `308` before any auth, in every environment, `loopback /health` excepted for systemd probes).
- **Per-tenant credential encryption** with a split-knowledge requirement:

```
KEK = HKDF-SHA256(  
    ikm    = VELAFLOW_CREDENTIAL_PEPPER,  
    salt   = SHA256(tenant_id || 0x1F || owner_google_sub),  
    info   = "velaflow-credential-v2",  
    length = 32
```

```
)
ciphertext = AES-256-GCM(KEK, plaintext, aad = field_name)
```

Decrypting a tenant's Todoist / Notion / Gmail / Gemini token requires **both** the operator pepper **and** the tenant's Google OAuth sub. A root shell inside the host with only one of the two retrieves nothing. Ciphertext cannot be relocated between tenants or between fields — the AAD and the per-tenant salt both reject it.

- **Deterministic digest, AI-optional.** A parameter-free scoring algorithm (Bronze → Silver → Gold) ranks tasks before any LLM runs. The platform produces a usable daily digest even when every upstream LLM is unavailable. The LLM chain (Gemini 2.5 Pro → Flash → Flash-Lite → Groq llama-3.3-70b → optional local Ollama) is a transparent enrichment layer, not a hard dependency.
- **Per-tenant scheduler** (`brain.queue.scheduler`) queues each tenant's jobs at their configured time and timezone; **per-tenant quotas** are enforced at the API layer (not the GUI) and survive worker restarts.
- **KEDA 0→10 workers** on queue depth, HPA 1→4 API pods, independent 0→3 premium GPU pool. LXC deployment simply pins all three to 1.
- **HMAC-SHA256-chained action ledger:** every ingest, LLM call, delivery, and configuration change is appended to a tamper-evident JSON-Lines record. `verify_chain()` returns the first broken offset offline.
- **Preflight refuses to boot a bad environment.** `scripts/preflight.py` is an `ExecStartPre=` for every systemd unit; a missing master key, a missing credential pepper, a reused JWT secret, or a missing TLS pair blocks startup before a single request is served.

What happens to operators who do not adopt a boundary like

this one

These failure modes are not hypothetical — the public post-mortems are named inline.

- **Root-shell-decrypts-everything.** Any multi-tenant productivity stack whose decryption key lives entirely in the operator environment is one compromised backup, one leaked `.env`, or one insider away from a mass credential dump. **LastPass, 2022:** the attacker lifted a backup plus the single KMS key and decrypted the vaults offline. Replicate that pattern at VelaFlow scale and every tenant's Todoist, Notion, Gmail, and Gemini tokens leave with the backup.
- **Plaintext-by-one-misconfig.** A single missing HSTS header or a middleware registered in the wrong order demotes the whole fleet to cleartext during a window nobody notices. **Okta support case upload, 2023:** one misused file, one misconfigured bucket, weeks of exposure. VelaFlow's `HTTPSOnlyMiddleware` is the outermost ASGI wrapper in every environment, including local dev, precisely to make that misconfiguration impossible.
- **Cross-tenant-ORM-leak.** Row-level filters without a cryptographic or schema-level boundary leak on the first malformed query, the first missing `WHERE`, the first middleware bypass. **Multiple published SaaS post-mortems, 2022–2025.** VelaFlow binds tenant identity into the credential KDF salt and into GCM AAD — an off-tenant query does not just return filtered-zero rows, it fails authentication.
- **Vendor-outage-equals-platform-outage.** Hard dependency on a single LLM provider means every model deprecation, regional outage, or price change is a tenant-visible incident. VelaFlow's

fallback chain plus the deterministic Gold layer guarantee a digest is produced even under full LLM provider blackout.

The secret formula (specs, not slogans)

Control	Specification
Credential KEK	<code>HKDF-SHA256(ikm=pepper, salt=SHA256(tenant_id \\ 0x1F \\ owner_google_sub), info="velaflow-credential-v2", length=32)</code>
Credential cipher	<code>AES-256-GCM, 96-bit nonce, 128-bit tag, aad = field_name</code>
Memory pinning	<code>mlockall(MCL_CURRENT MCL_FUTURE)</code> at process start on API + worker; <code>LimitCORE=0, LimitMEMLOCK=infinity</code> on all six brain-*.service units
HTTPS enforcement	<code>HTTPSOnlyMiddleware</code> outermost in every environment; <code>HTTP → 308</code> ; <code>loopback /health</code> exempted for systemd probes
Static-analysis floor	Snyk Code (SAST) 0 findings across HIGH/MEDIUM/LOW with 0 .snyk ignores ; <code>pip-audit</code> (PyPA-authoritative SCA) 0 advisories; Bandit 0 medium/high
Regression floor	<code>pytest tests/ --ignore=tests/test_stress.py → 528 passed</code> ; any regression fails the build
Pentest floor	58 adversarial tests across 19 audit rounds; every finding remediated or formally mitigated with documented evidence
Boot-time gate	<code>scripts/preflight.py</code> as <code>ExecStartPre=</code> ; a bad environment blocks startup before any request is served
Tamper evidence	HMAC-SHA256-chained JSONL action ledger; <code>verify_chain()</code> returns the first broken offset offline

Each row above is backed by a test that fails if the control regresses. Nothing in this section is aspirational.

3. Familiar-stack translation

Readers arriving from a large cloud data-and-AI stack will recognise most of the patterns below. VelaFlow deliberately substitutes self-hosted, commodity-friendly equivalents — the goal is a single VPS or a small LXC on a homelab, not a €40k/month managed bill. The intent of each choice is the same as the cloud-native version; the cost envelope and the operational surface are very different.

Pattern / tool in the large-cloud world	Where it lives in VelaFlow	Why this substitution
Databricks Lakehouse (workspace-managed Spark + Delta)	DuckDB analytical engine in <code>src/brain/engine/</code> + SQLite per-tenant catalog in <code>src/brain/catalog/</code>	Single-node, embedded, zero-ops; a SaaS for 1–1 000 paying tenants does not need a distributed cluster until it measurably does. The engine boundary is abstract, so Spark/Delta can be slotted in without rewriting the pipeline.
Delta Lake / Apache Iceberg (ACID table format, time travel)	DuckDB tables + append-only HMAC-chained action ledger	Tamper-evident history across all tenant-write paths without requiring an external metastore. Full Delta semantics are not needed at this tenant count.
Unity Catalog (metastore, RBAC, lineage, audit)	<code>src/brain/catalog/</code> (local catalog) + <code>src/brain/security/rbac.py</code> + <code>src/brain/security/action_ledger.py</code>	RBAC is enforced at the API + tenant-manager boundary; lineage is captured in the action ledger; audit is the HMAC-chained JSON-Lines stream. Offline-verifiable, single operator.
Mosaic AI / Vertex AI / Bedrock (managed LLM orchestration, RAG)	<code>src/brain/llm/</code> multi-model fallback + <code>src/brain/rag/</code> (chunker, embedder, DuckDB-backed vector store)	Avoids vendor lock-in; every tenant pays for only the upstream LLM minutes they use; Premium/VIP tiers can run fully local with Ollama.
Pinecone / Weaviate / Qdrant (managed vector DB)	Embedded vector store over DuckDB, per-tenant partitioned	One fewer network hop; per-tenant isolation is a row filter, not an index re-provision.
LangChain / LlamaIndex orchestration frameworks	Thin, explicit orchestration in <code>src/brain/pipeline/</code> with versioned prompt templates under <code>prompts/</code>	The platform does not need a general graph runtime; it needs five deterministic pipeline stages with clear contracts. Prompt provenance is captured in the ledger.
Apache Airflow / Dagster / Prefect (workflow orchestration)	<code>src/brain/queue/scheduler.py</code> — per-tenant cron loop + in-process queue	Scales horizontally behind Redis + KEDA when required; does not require a standing scheduler container for the single-operator deployment.
Kafka / Pulsar / Flink / Kinesis (streaming ingress + stream processing)	In-process <code>TaskQueue</code> with at-most-once idempotency keys; Redis swap-in documented for	Same backpressure + retry semantics, no broker to run for a tenant count under

Pattern / tool in the large-cloud world	Where it lives in VelaFlow	Why this substitution
	HA; DuckDB streaming ingestion on the Bronze layer	four digits. Stream processing stays Python-native rather than JVM.
Kubernetes + KEDA + HPA (event-driven autoscaling)	Same — <code>deploy/k8s/</code> includes KEDA <code>ScaledObject</code> manifests for the worker; HPA for the API; GPU pool on a separate node-selector	Identical pattern, same manifests. The LXC path is simply “scale-to-one”.
Databricks workspace admin (cluster policies, network isolation, Private Link, VNet injection, IP allow-lists)	<code>systemd</code> sandboxing (<code>NoNewPrivileges</code> , <code>ProtectSystem=strict</code> , <code>CapabilityBoundingSet=~CAP_SYS_ADMIN</code> , <code>LockPersonality</code> , <code>RestrictRealtime</code>), loopback-only Prometheus/Grafana, mandatory reverse-proxy-with-auth for any external access	The same posture — “this process has the minimum privilege and the minimum network reach required to do its job” — enforced at the unit-file layer rather than at the cloud-control-plane layer.
Databricks RBAC at workspace/catalog/object level	<code>src/brain/security/rbac.py</code> enforced at the API + tenant-manager boundary; <code>owner_google_sub</code> is first-class on the tenant row	Role resolution happens once per request and is bound to the Google OAuth subject claim; no in-band privilege escalation path.
Azure Key Vault / AWS KMS / HashiCorp Vault	<code>src/brain/security/encryption.py</code> (<code>FieldEncryptor</code> for at-rest data, <code>CredentialEncryptor</code> for third-party tokens) with operator pepper and per-tenant <code>owner_google_sub</code> binding	Envelope encryption with a split-knowledge requirement (pepper + owner sub). Rotating the pepper re-wraps every credential; no external KMS quota to budget.
Google Cloud IAM / Azure AD B2C (managed identity)	Mandatory Google OAuth via <code>src/brain/api/routes/auth.py</code> ; no password auth exposed externally; <code>owner_google_sub</code> is a first-class column on the tenant row	The identity provider is the one the tenant already trusts; VelaFlow never stores a password hash, so there is nothing to leak.
Snyk + Dependabot + CodeQL in a managed CI	Local gates: Snyk Code + SCA, Bandit, pip-audit, pytest, preflight — all required to return zero before release	The same bar, enforced locally and in CI. See the Engineering Status table.
Datadog / New Relic APM	Prometheus metrics on <code>127.0.0.1:9090</code> + Grafana on <code>127.0.0.1:3000</code> , behind a reverse	Operator-owned, loopback by default; no customer data leaves the host to a third-party SaaS.

Pattern / tool in the large-cloud world	Where it lives in VelaFlow	Why this substitution
	proxy with auth for remote access	
Terraform / Pulumi / Helm	<code>docker-compose.yml</code> , <code>deploy/k8s/</code> manifests, and <code>scripts/install.sh</code> ; Helm charts are a v1.1 packaging step	The platform targets Proxmox LXC + Compose as the primary operator experience. Terraform can wrap the k8s manifests for multi-cloud; intentionally out of scope for v1.0.
MLOps: model versioning, drift monitoring, automated retraining, model serving	Versioned prompt templates under <code>prompts/</code> ; model-id recorded per request in the action ledger; per-tenant fallback chain records which model served each call; drift is detected via downstream KPI checks on Gold outputs	VelaFlow consumes frontier LLMs as a dependency rather than training its own; the MLOps surface is limited to prompt versioning, model selection, and outcome telemetry. Training-pipeline tooling is intentionally out of scope.
Fraud-detection / anomaly-detection on real-money streams	Out of scope for v1.0. The scheduler, queue, and Gold-layer scoring algorithm are the correct substrate for it, but the platform has no financial-transaction domain model	Named here for honesty — a v1.4+ connector could plug a financial stream into the Bronze ingest without platform changes.
Geospatial / raster / GDAL / COG pipelines	Out of scope. VelaFlow ingests text-shaped personal-productivity data, not imagery	Named for honesty; a geospatial fork would replace the Bronze ingest and the DuckDB schema, not the security or scheduling substrate.
SAP / S/4HANA / BW/4HANA extractors	Out of scope. VelaFlow ingests from Todoist, Notion, Google Calendar, Gmail, and user-uploaded documents	The ingestion layer is pluggable (<code>src/brain/connectors/</code>) so additional sources are a small connector, not a platform rewrite.

The pattern the reader should see is: the **shape** of each responsibility (catalog, credential isolation, autoscaling, lineage, multi-tenant boundaries, RBAC, identity, observability) is the same one a managed cloud stack would use. The **realisation** is chosen to fit a single operator on commodity hardware, with a deliberate migration path for each component once the tenant base justifies it. Rows explicitly marked *out of scope* are named so the reader does not have to guess.

4. Product Overview

Capabilities

Capability	Description
Unified task ingest	Todoist API v1 with cursor pagination; optional Google Calendar OAuth2; optional Gmail IMAP triage.
Deterministic prioritisation	A parameter-free scoring algorithm ranks tasks before any LLM touches them. The platform produces a usable digest even when every AI backend is unavailable.
Multi-model LLM fallback	Gemini 2.5 Pro → Flash → Flash-Lite → Groq llama-3.3-70b. The chain transparently handles rate limits, outages, and quota exhaustion.
Private LLM (paid tiers)	Ollama-based local inference with GPU auto-detection and CPU fallback. Default model fits 4 GB.
Retrieval-augmented generation	Per-tenant document store with chunked embeddings and vector search; 500 documents on Premium, 5,000 on VIP.
Two-way Notion ↔ Todoist sync	Edits in either system are reconciled without data loss.
Delivery channels	Email (SMTP), Notion page update, optional WhatsApp via CallMeBot.
Per-tenant scheduling	A multi-tenant cron scheduler queues each tenant's jobs at the times they configure, honouring their timezone.
Tenant self-service GUI	Streamlit surface gated by tier; controls above the tenant's tier are disabled and the API enforces the same rules.
Billing	Stripe Checkout + webhooks for all paid tiers; subscription state is the source of truth for tier.
Encrypted backups	Service-account Google Drive uploads with envelope encryption; zero-touch restore.
Admin dashboard	Prometheus metrics, Grafana dashboard, HMAC-chained action ledger.

How a typical day runs on VelaFlow

1. The per-tenant scheduler fires the tenant's daily digest at their configured time and timezone.
2. The worker decrypts that tenant's third-party credentials **in memory only**, constructs a per-request `Settings` object, and runs the Bronze → Silver → Gold pipeline inside an isolated task.
3. The Gold layer produces the digest; delivery respects the tenant's channel toggles (email, Notion, WhatsApp).
4. Every action — ingest, LLM call, delivery, configuration change — is appended to an HMAC-chained JSON-Lines ledger scoped to that tenant.
5. Daily usage counters are persisted so quota enforcement survives worker restarts.

5. Subscription Tiers

Enforcement is always at the API layer. The GUI is a convenience surface; it does not grant capabilities the API would refuse.

Capability	Free	Standard	Premium	VIP
Todoist + Notion two-way sync	Fixed schedule	Configurable	Configurable	Configurable
Daily digest email	✓	✓ custom time	✓ custom time	✓ custom time
Pipeline runs / day	3	20	100	999
Tasks tracked	100	1,000	10,000	50,000
LLM calls / day	5	50	200	999
Storage quota	50 MB	500 MB	5 GB	10 GB
WhatsApp + overdue alerts	—	✓	✓	✓
Gmail IMAP triage	—	✓	✓	✓
Google Calendar context	—	✓	✓	✓
Weekend planner + weekly review	—	✓	✓	✓
Local LLM (Ollama)	—	—	✓	✓
Premium cloud LLM (Gemini Pro)	—	—	✓	✓
NotebookLM synchronisation	—	—	✓	✓
Native on-box RAG (/api/v1/rag/*)	—	—	—	✓ 5,000 / 500 q/day
Bring-your-own gemini_api_key	—	—	✓	✓
Priority support	—	—	—	✓

Roles

Role	Scope
Admin / Owner	Platform operator. Auto-provisioned on first Google OAuth login from the configured owner email. Granted the VIP tier plus platform permissions.

Role	Scope
Demo	Time-boxed tenant created by <code>scripts/seed_demo.py</code> or an admin action. Read-only for sensitive scopes; auto-expires.
Free / Standard / Premium / VIP	End-user tenants. Tier is set by Stripe webhooks after checkout; downgrade on cancellation.

All non-owner tenants authenticate with Google OAuth. Password-based authentication is not exposed externally; the legacy `POST /api/v1/tenants` endpoint remains only for programmatic bootstrap and always creates a Free tenant regardless of email.

6. Security and Privacy Model

VelaFlow's threat model assumes an attacker may already hold shell access inside the host. Every filesystem-touching code path is layered so a local shell cannot induce the application to write, read, `chmod`, or extract outside an allow-listed set of directories, and no tenant credential is persisted in cleartext.

Static-analysis posture

Scanner	Scope	Result
Snyk Code (SAST)	Full Python tree	0 findings at medium+, 0 . snyk ignores
pip-audit (SCA, PyPA-authoritative)	Installed Python packages	0 known advisories
Bandit	<code>src/ + scripts/</code>	0 medium or high

Key controls

- **`src/brain/security/safe_path.py`** — every untrusted path passes through an allow-list (`VELAFLOW_DATA_DIR`, process HOME, `cwd`, `/var/log/brain` on POSIX, `%PROGRAMDATA%\brain` on Windows) before any filesystem operation.
- **Inline sink sanitisation** — every filesystem sink re-validates the resolved path with `Path.resolve().relative_to(base)` immediately before the call. A symlink TOCTOU attempt hits the sanitiser a second time.
- **File-descriptor APIs over path-string APIs** — `os.fchmod(fd, ...)` on the already-open stream, `os.open(path, ..., mode=0o600)` at creation, `pathlib.Path.open('a')` for the action ledger. `Chmod-by-path` sinks are eliminated rather than merely guarded.
- **Hardened archive restore** — the backup restore path uses `tar.extractfile() + shutil.copyfileobj(..., length=64 * 1024)` with per-member and per-write containment checks. `tar.extract(path=...)` is not used.
- **Per-tenant field encryption** — AES-256-GCM with PBKDF2-derived keys per tenant, used for non-credential data at rest (e.g. webhook signing keys). The platform master key is held in the operator environment only (`VELAFLOW_MASTER_KEY`).
- **Credential vault (third-party API keys)** — from Round 18, tokens for Todoist, Notion, Gmail, LiteLLM, Gemini and Google OAuth are encrypted under a key derived from HKDF-

SHA256(ikm=pepper, salt=SHA256(tenant_id || 0x1F || owner_google_sub), info="velaflow-credential-v2") with AES-256-GCM and the field name passed as authenticated associated data. The pepper (VELAFLOW_CREDENTIAL_PEPPEER) must differ from the master key. An operator with shell access alone cannot decrypt a tenant's credentials without **both** the pepper **and** the tenant's Google OAuth subject claim. Ciphertext cannot be relocated between tenants or between fields.

- **HTTPS-only, always** — `HTTPSOnlyMiddleware` is the outermost middleware in every environment (including local dev); plain-HTTP requests receive **308 Permanent Redirect** before any auth or tenant resolution runs. `X-Forwarded-Proto: https` from a trusted reverse proxy is honoured; the only cleartext exception is the loopback `/health` family for `systemd ExecStartPost` probes.
- **HMAC-chained telemetry** — `src/brain/security/action_ledger.py` writes an append-only, HMAC-chained JSON-Lines record. Any tamper attempt breaks the chain and is detectable offline.
- **Systemd sandboxing** — units set `NoNewPrivileges=yes`, `ProtectSystem=strict`, `ProtectHome=read-only`, `CapabilityBoundingSet=~CAP_SYS_ADMIN`. From Round 19, every `scripts/brain-*.service` additionally sets `LimitCORE=0` (no core dumps), `LimitMEMLOCK=infinity` (allow `mlockall`), `LockPersonality=true`, `RestrictRealtime=true`, `ProtectClock=true` and `ProtectHostname=true`. The API and worker call `mlockall` (`src/brain/security/memlock.py`) at startup so decrypted credential pages are pinned in RAM and never written to swap.
- **Loopback-only observability** — Prometheus and Grafana bind to `127.0.0.1`; external access must go through a reverse proxy with auth.
- **Mandatory Google OAuth** — there is no password login exposed to external users.

See `docs/SECURITY-AUDIT.md` for the full audit history and `docs/security.md` for operational guidance.

7. Engineering Status

Signal	Status
Test suite	528 / 528 passing (pytest tests/ --ignore=tests/test_stress.py)
Snyk Code (SAST)	0 findings across HIGH / MEDIUM / LOW , 0 ignores, no .snyk
pip-audit (SCA — PyPA-authoritative)	0 known advisories against the resolved dependency graph (<code>pip install --dry-run -r requirements.txt</code> confirms <code>cryptography==46.0.7</code> is the resolved version)
Bandit	0 medium / high
Preflight (<code>scripts/preflight.py</code>)	0 blocking, 0 warnings (with master key, credential pepper, JWT secret, Google OAuth client id/secret, TLS cert + key configured)
HTTPS enforcement	Always-on: plain HTTP is 308-redirected to HTTPS in every environment, loopback <code>/health</code> excepted for <code>systemd</code> probes

Signal	Status
Credential model	Third-party credentials encrypted under <code>HKDF(pepper \\ SHA256(tenant_id\ owner_google_sub))</code> with AES-256-GCM and field-name AAD; operator shell access alone cannot decrypt without both the pepper and the tenant's owner sub
Dependency pinning	<code>cryptography>=46.0.7</code> and the rest of the critical path pinned in <code>requirements.txt</code>
Pentest history	58 adversarial tests across 19 audit rounds (R14 baseline + R17 tenant isolation + R18 credential vault / HTTPS + R19 zero-LOW scrub + memory-dump hardening + credential-in-transit end-to-end); every finding remediated or formally mitigated
Memory-dump hardening	<code>mlockall(MCL_CURRENT MCL_FUTURE)</code> at process start, <code>LimitCORE=0</code> , <code>LimitMEMLOCK=infinity</code> , <code>LockPersonality</code> , <code>RestrictRealtime</code> , <code>ProtectClock</code> , <code>ProtectHostname</code> on all six <code>brain-*.service</code> units — decrypted credentials cannot reach swap or a core file

8. Architecture (Summary)

VelaFlow follows a medallion data architecture per tenant.

```

Third-party APIs → [ Bronze ] → [ Silver ] → [ Gold ] → API / Digest / Notion
                  Raw land   Clean + dedup   Score +
                  Tenant-    + PII mask      AI-enrich
                  partitioned

```

- **Bronze:** raw API payloads, tenant-partitioned, stored in DuckDB.
- **Silver:** schema validation, dedup by business key, PII masking.
- **Gold:** deterministic scoring and AI-enriched digest production.

The runtime is structured as:

- **FastAPI** application factory (`brain.api.app:create_app`) on port 8765.
- **Queue + worker** (`brain.queue.worker`) running per-tenant jobs.
- **Tenant scheduler** (`brain.queue.scheduler`) — a cron-like loop that scans all active tenants and enqueues their scheduled jobs.
- **DuckDB analytical engine** and **SQLite data catalog**, both local.
- **Observability:** Prometheus + Grafana on loopback.
- **Billing:** Stripe Checkout + webhooks; subscription events drive tier changes.

Full diagrams and per-layer detail are in `docs/architecture.md`.

9. API Surface

Selected endpoints. All authenticated endpoints require a JWT issued by the Google OAuth flow. Tier-gated endpoints return **403 with an upgrade_to field** when the tenant's tier is insufficient.

Endpoint	Method	Description
/health	GET	Live / ready probes
/metrics	GET	Prometheus text format
/api/v1/auth/google	GET/POST	Google OAuth flow (owner and tenants)
/api/v1/tenants/me	GET	Current tenant profile
/api/v1/tenants/me/config	PATCH	Update schedule, delivery, tokens (tier-gated)
/api/v1/pipelines/run	POST	Trigger Bronze→Silver→Gold pipeline
/api/v1/tasks/scored	GET	Retrieve scored tasks (Gold layer)
/api/v1/digests/daily	GET	Retrieve daily digest
/api/v1/rag/ingest	POST	Ingest a document into the per-tenant DuckDB VSS vector store (VIP only — premium keeps NotebookLM)
/api/v1/rag/query	POST	Retrieval-augmented query over the tenant's own vectors (VIP only)
/api/v1/rag/stats	GET	Document count and per-tier quota (VIP only)
/api/v1/rag/documents/{id}	DELETE	Remove a document and all its chunks (VIP only)
/api/v1/billing/checkout	POST	Create Stripe checkout session
/api/v1/webhooks/stripe	POST	Stripe subscription webhook (public)
/api/v1/dashboard/overview	GET	Dashboard payload (connections, config, usage)
/api/v1/vault/keys	GET / POST	Encrypted per-tenant key vault

The full surface is documented in docs/architecture.md.

10. Deployment

The production surface is **declarative Terraform**. `scripts/install.sh` remains available as a developer quick-start and is deliberately labelled dev-only.

Zero-cost hosting is a project invariant — today. VelaFlow's public service must cost the maintainer nothing at steady state. Every **shipped** deployment target is **free forever** at the provider tier we pin: the homelab target uses owned hardware, `generic-vm` reuses an already-paid host, and Oracle Cloud uses OCI's **Always Free** Ampere A1.Flex tier (4 OCPU / 24 GB RAM, no expiry). Paid clouds (AWS, Azure, GCP, Databricks) are deliberately **not** shipped as deployment

targets in this release — they would quietly break the free-forever promise if an operator picked one by accident. They remain **open, documented future options**: if user demand outgrows what free tiers can serve, a paid-cloud target can be added and activated only when paid VIP revenue covers the incremental bill (see docs/scaling-path.md for Stage 1 / Stage 2). The only paid surface that exists today is the **VIP** user tier, priced strictly above its marginal cost.

VelaFlow’s deploy contract is *portable host IaC*: the same Terraform module installs an identical VelaFlow stack on any of these free Linux hosts. Three equal-status target directories live under `deploy/terraform/` — no cloud is “primary”:

Target	Host type	Cost
<code>deploy/terraform/proxmox/</code>	Unprivileged LXC on Proxmox VE (homelab, Intel N95 / 8 GB — the hardware floor)	€0 — self-hosted hardware
<code>deploy/terraform/generic-vm/</code>	Any SSH-reachable Linux host already owned (home server, old PC, LXC guest)	€0 when reusing existing capacity
<code>deploy/terraform/oracle-cloud/</code>	OCI Always Free Ampere A1.Flex VM (4 OCPU / 24 GB RAM, free forever)	€0 — forever

All three delegate to the shared module `deploy/terraform/modules/velaflow-host/`, which renders a cloud-init document (Podman + systemd unit + nginx TLS reverse proxy + data-dir permissions) and either pushes it via SSH remote-exec (proxmox, generic-vm) or hands it to the caller’s VM as `user_data` (oracle-cloud). Either way the VelaFlow install itself is strictly declarative.

Preflight

Before `terraform apply`, validate the target structurally (no Terraform CLI required):

```
python scripts/terraform_preflight.py deploy/terraform/proxmox
python scripts/terraform_preflight.py deploy/terraform/generic-vm
python scripts/terraform_preflight.py deploy/terraform/oracle-cloud
```

The same checks run under pytest via `tests/test_terraform_modules.py`, so a malformed `.tf` fails the CI suite. All three targets are additionally validated with the real terraform CLI (`fmt -check`, `init -backend=false, validate`) on Terraform v1.6+ — see docs/SECURITY-AUDIT.md Round 21.

Dev quick-start (not production)

```
scripts/install.sh          # Bash bootstrap on a single LXC / laptop
docker compose up -d       # docker-compose.yml for local dev
```

Scaling out

Kubernetes + KEDA manifests in `deploy/kubernetes/` are the Stage 2 horizontal-scale surface (see docs/scaling-path.md); they are **not** part of this IaC tier because they dwarf the hardware floor. Rationale: docs/adr/0003-terraform-iac-vs-bash-install.md.

Minimum configuration

```
export VELAFLOW_MASTER_KEY=$(python -c "import
os,base64;print(base64.urlsafe_b64encode(os.urandom(32)).decode())")
export JWT_SECRET=$(python -c "import secrets;print(secrets.token_urlsafe(48))")
export GOOGLE_OAUTH_CLIENT_ID="..."
export GOOGLE_OAUTH_CLIENT_SECRET="..."
export VELAFLOW_OWNER_EMAIL="you@example.com"
python scripts/preflight.py # must report 0 blocking
python -m uvicorn brain.api.app:create_app --factory --host 127.0.0.1 --port 8765
```

See docs/deployment.md for full step-by-step instructions, Kubernetes manifests, Grafana dashboards, and backup setup.

11. Roadmap

- **v1.0 (current)** — API, auth, tier enforcement, tenant scheduler, Streamlit GUI, Stripe billing, dashboard endpoints, demo seeder, encrypted Drive backups, HMAC-chained action ledger.
- **v1.1** — PostgreSQL tenant registry for multi-node writes. Redis queue as the recommended backend past a single worker.
- **v1.2 (headline)** — a per-tenant graphical, drag-and-drop workflow editor. This replaces the current Streamlit surface as the primary personalisation interface.
- **v1.3** — webpage-monitor connector, Gmail-topic watcher.

n8n Community Edition and the in-process queue are treated as provisional v1.0 surfaces. The platform does not depend on either for its core pipelines; both are swappable.

12. Repository Layout

```
.
├─ src/brain/           Application code
│  ├─ api/             FastAPI routes and middleware
│  ├─ queue/           Worker + tenant scheduler
│  ├─ tenant/          Models, manager, quotas
│  ├─ security/        safe_path, action_ledger, secure_logging, rbac
│  ├─ engine/          DuckDB pipeline (Bronze → Silver → Gold)
│  ├─ catalog/         SQLite data catalog
│  ├─ rag/             Chunker, embedder, vector store
│  └─ gui/             Streamlit tier-gated self-service surface
├─ scripts/
│  ├─ preflight.py      Pre-deployment validator
│  ├─ seed_demo.py      Create time-boxed demo tenants
│  ├─ drive_backup.py   Service-account Drive backup + restore
│  └─ build_pdfs.py     Regenerate technical-reference PDFs
├─ tests/              528 pytest cases
├─ deploy/
│  ├─ docker/          Dockerfile + Compose
│  ├─ kubernetes/      Manifests + KEDA ScaledObject
│  ├─ lxc/             Proxmox LXC helpers
│  ├─ observability/   Grafana dashboards, Prometheus config
│  └─ terraform/       Portable host IaC (proxmox + generic-vm + oracle-cloud via
shared module)
└─ docs/
```

```
| |─ architecture.md
| |─ architecture-enterprise.md
| |─ deployment.md
| |─ scaling-path.md
| |─ security.md
| |─ SECURITY-AUDIT.md
| |─ adr/           Architecture Decision Records (0001, 0002, 0003)
└─ config/pipeline.yaml Declarative per-stage configuration
```

13. Authorship and provenance

Written and maintained by one person. Inception date: **2026-04-17**.

Measured at the current HEAD:

- `src/` — ~**15 000** lines of application code (R21: native RAG API, portable IaC)
- `tests/` — ~**6 000** lines of tests (**528 passing**, 0 ignores)
- `scripts/` — ~**3 500** lines of operational scripts and validators
- `docs/` — ~**9 500** lines of technical documentation

The code was authored with heavy use of GitHub Copilot inside VS Code. Every architectural decision, every threat-model round, every control listed in Section 2.5 (*The secret formula — specs, not slogans*), and every remediation in `docs/SECURITY-AUDIT.md` is the maintainer's. The code is AI-assisted; the design, the boundaries, and the risk posture are not.

14. License

MIT. See LICENSE.

Technical README (docs/README-technical.md)

VelaFlow — Technical Reference

A multi-tenant AI productivity platform built on a **medallion architecture** (Bronze → Silver → Gold), integrating Todoist, Notion, Google Calendar, Gmail, and Google NotebookLM into one automated pipeline. Delivers daily task digests, AI-planned schedules, Kanban board intelligence, two-way data synchronisation, and automated knowledge-base updates — deployable as a hardened Proxmox LXC, Docker Compose stack, or Kubernetes cluster with KEDA auto-scaling.

Release: v1.0.0-rc1 (Round 18 — credential vault + HTTPS-always)

Verification at this release (see docs/SECURITY-AUDIT.md for methodology):

- pytest: 500/500 passing (tests/test_stress.py excluded from this gate; run nightly).
- Snyk Code: 0 HIGH / 0 MEDIUM findings.
- Snyk Open Source (SCA): 0 vulnerable paths across 45 dependencies.
- Bandit (-ll): 0 Medium / 0 High findings across 16 812 lines.
- pip-audit: 0 known vulnerabilities.
- Pentest suite: 32 adversarial tests (R14 + R17 + R18), all pass.
- Preflight gate: 0 blocking / 0 warnings with the documented production env.

Round 18 Changes (what's new since v1.0.0-rc1 R17.3)

Round 18 hardens the credential surface and makes HTTPS non-optional. The changes are deliberately narrow: no feature additions, no new dependencies, no API breakage for callers that already used HTTPS.

1. Credential vault refactor (CredentialEncryptor)

Third-party credentials (Todoist, Notion, Gmail OAuth, LiteLLM proxy, Gemini, Google refresh tokens) are no longer encrypted with the platform master key alone. They are now encrypted under a key derived from **three independent secrets**:

$H = \text{HKDF} - \text{SHA256} (ikm = \text{pepper}, \text{salt} = \text{SHA256}(\text{tenant_id} \parallel 0x1F \parallel \text{owner_google_sub}), \text{info} = \text{"velaflow-credential-v2"}, L = \dots)$

- **pepper** — VELAFLow_CREDENTIAL_PEPPER, a ≥32-byte base64 secret held only in the deployment's systemd EnvironmentFile=. Must differ from VELAFLow_MASTER_KEY.
- **tenant_id** — binds ciphertext to one tenant; a row copied to another tenant is unreadable (even by the platform operator).
- **owner_google_sub** — the Google OAuth subject claim of the tenant's owner. It is written exactly once (bind_owner_sub) on the first successful OWNER/ADMIN OAuth login and is refused thereafter if a different sub is presented.

Encryption is AES-256-GCM. The **field name** is passed as authenticated associated data (AAD), so a ciphertext encrypted as **todoist_api_token** cannot be relocated to **gemini_api_key** even within the same tenant.

Ciphertext layout: `base64url(0x02 || nonce(12) || ct || tag)` — the schema byte lets a future migration rotate the KDF without a big-bang backfill.

Threat model explicitly covered:

- Operator with root on the storage node has the ciphertext but not the pepper: cannot decrypt.
- Operator with pepper but not the per-tenant owner sub: cannot decrypt.
- Attacker who steals a database dump: needs both the pepper and every owner's sub.
- Ciphertext relocation (row-copy between tenants, field-copy within a tenant): rejected.
- Pepper rotation invalidates all existing credentials (intentional — forces re-entry).

The legacy `FieldEncryptor` (master-key + PBKDF2) is retained for **non-credential** data at rest (e.g. the per-tenant `api_key` used for webhook signing), where the owner-sub dimension would make self-service unusable.

2. HTTPS-only middleware (always-on)

`HTTPSOnlyMiddleware` now runs as the **outermost** middleware in every environment, including local development. A plain-HTTP request receives **308 Permanent Redirect** to the `https://` form of the same URL before any authentication, tenant resolution, or CORS logic runs.

- **X-Forwarded-Proto:** `https` from a trusted reverse proxy (Caddy, nginx) is honoured — TLS termination upstream is still correct.
- The narrow loopback exemption (`127.0.0.1 / ::1 / localhost`) applies **only** to the three health endpoints (`/health`, `/health/live`, `/health/ready`) and **only** when no `X-Forwarded-Proto` header is present, to avoid breaking systemd `ExecStartPost` probes that run before TLS material is available.

3. Preflight tightened (blocking env)

`scripts/preflight.py` now treats the following as **blocking** (exit 1 until set):

Variable	Requirement
<code>VELAFLOW_MASTER_KEY</code>	base64, decodes to ≥ 32 bytes
<code>VELAFLOW_CREDENTIAL_PEPPER</code>	base64, decodes to ≥ 32 bytes, must differ from master key
<code>JWT_SECRET</code>	≥ 32 characters
<code>GOOGLE_OAUTH_CLIENT_ID</code>	present
<code>GOOGLE_OAUTH_CLIENT_SECRET</code>	present
<code>VELAFLOW_TLS_CERT</code>	set, file exists, readable
<code>VELAFLOW_TLS_KEY</code>	set, file exists, readable

The only informational-but-not-blocking checks left in preflight are `VELAFLOW_DATA_DIR`, `VELAFLOW_LOG_HMAC_KEY`, and `VELAFLOW_DISABLE_OPEN_REGISTRATION` — three operator choices with safe defaults.

4. Pentest suite (tests/test_pentest_r18.py)

Twenty new adversarial tests, grouped into three classes:

- `TestCredentialEncryptorPrimitives` (12 tests) — pepper validation, AAD binding, tenant / field / owner-sub relocation rejection, schema-version rejection, byte-flip authenticated-decryption failure, master-key isolation, pepper rotation.
- `TestOwnerSubBinding` (4 tests) — idempotent bind, refuse-rebind-to-different-sub, credentials refused before bind, decrypt refused when bind is missing.
- `TestHTTPSOOnlyMiddleware` (3 tests) — cleartext protected routes are 308-redirected, `X-Forwarded-Proto: https` is honoured, loopback `/health` is exempt (verified via a hand-built ASGI scope because `TestClient` reports a non-loopback client host).

Overview

Modern knowledge workers operate across multiple disconnected tools. Tasks sit in Todoist, events in Google Calendar, emails in Gmail, project notes in Notion, and research in Google NotebookLM. The cost of context-switching between these systems — and the cognitive overhead of manually deciding what to prioritise — is substantial.

VelaFlow eliminates that overhead by acting as an automated coordination layer:

1. **Reads** all active tasks (Todoist REST API v1 with cursor pagination), calendar events (Google Calendar OAuth2), and unread emails (Gmail IMAP).
2. **Scores** every task against a deterministic, parameter-free priority algorithm — no AI required for ranking.
3. **Generates** a polished daily plan via a multi-model LLM fallback chain (Gemini 2.5 Pro → Flash → Flash-Lite → Groq llama-3.3-70b).
4. **Delivers** the digest by email (SMTP), WhatsApp (CallMeBot), and directly into Notion.
5. **Syncs** task state bidirectionally between Todoist and Notion — edits in either tool are reconciled without data loss.
6. **Pushes** all knowledge (Notion pages + Todoist active tasks) into a Google NotebookLM notebook via Playwright-based browser automation.

The system requires no manual intervention once deployed. All AI uses free-tier endpoints routed through a self-hosted proxy. Total additional infrastructure cost: **\$0/month**.

Key Design Decisions

Decision	Rationale
Zero-Trust Proxy over secrets vault	Real AI keys never enter the LXC. A budget-capped LiteLLM proxy token is the only credential — theft costs \$1-2, not a full billing account. Revoke in one curl call. See docs/security.md.
Deterministic scoring before LLM	Tasks are ranked by a parameter-free algorithm first. AI only polishes the output. If every AI provider fails, the system still delivers a usable digest.
systemd over cron	Structured logging via <code>journalctl</code> , dependency ordering, sandboxing (<code>NoNewPrivileges</code> ,

Decision	Rationale
	<code>ProtectSystem=strict</code>), automatic restart on failure.
Two-way Notion sync over one-way	Users edit tasks in either Todoist or Notion. Sync detects drift and reconciles without data loss or duplication.
Multi-model fallback chain	Gemini 2.5 Pro → Flash → Flash-Lite → Groq llama-3.3. Each step is cheaper/faster. The chain handles rate limits, outages, and quota exhaustion transparently.
LXC over Docker-only	Lighter resource footprint, native systemd, direct Proxmox integration. Docker runs inside for optional n8n.
Unofficial NotebookLM API	No official API exists. Uses <code>notebooklm-py</code> (Playwright browser automation) with cookie auth. Headless auth solved via Xvfb + x11vnc in LXC.
Cursor-paginated API client	Todoist API v1 uses cursor pagination for large task sets. The client handles pagination transparently and includes delete-protection safeguards.
Medallion architecture (Bronze→Silver→Gold)	Standard data engineering pattern. Raw data (Bronze) is immutable. Cleaning, dedup, PII masking (Silver) run separately from scoring/enrichment (Gold). Each layer is independently testable, replayable, and auditable.
JWT over OAuth2 / external IdP	Minimal dependencies. Custom HS256 implementation — no PyJWT or external auth provider required. Suitable for self-hosted single-operator deployments.
KEDA over static replicas	Workers scale to zero when idle (cost savings) and burst to 10 on queue depth. Premium pods with GPU scale 0-3 independently. Appropriate for compute-intensive LLM workloads on the premium tier.
Per-tenant field encryption	PBKDF2-derived keys per tenant, AES-256-GCM authenticated encryption. Compromise of one tenant's data does not expose others.
In-process queue (upgradeable to Redis)	No external dependency for single-node deployments. Queue interface is abstract — swap to Redis for multi-node. Dead letter support for failed messages.
DuckDB analytical engine	Self-hosted on-prem analytical engine. Same medallion architecture as managed lakehouses, SQL-native transforms, 512 MB memory budget. No cloud dependency.

Decision	Rationale
SQLite catalog	On-prem data catalog. Namespace/schema/table registry, RBAC grants, data lineage tracking. WAL mode, secure_delete, parameterised queries only.
YAML pipeline config	Declarative <code>config/pipeline.yaml</code> — stages, grants, engine settings, and n8n webhook endpoints.
n8n per-tenant orchestration	Each tenant customises their pipeline schedule, data sources, and delivery channels via n8n’s visual editor — no coding required. Webhook endpoints allow async triggers.
Local LLM (premium tier)	Ollama-based inference for privacy-first tenants. GPU auto-detection with CPU fallback. Default model: qwen2:1.5b (fits 4 GB). Scale up by changing one env var.

Feature Set

Feature	Schedule	Delivery	Status
Daily Briefing	Mon-Fri 07:00	Email + Notion	Tested
Daily Briefing	Mon-Fri 07:00	WhatsApp (optional)	Requires CallMeBot setup
Overdue Alerts	Every 4 h (08-20)	WhatsApp (optional)	Requires CallMeBot setup
Weekend Planner	Friday 17:00	Email + Notion	Implemented
Weekly Review	Sunday 20:00	Email + Notion	Implemented
Board Analysis	On demand	Terminal	Implemented
Board Organise	On demand	Todoist (move + label tasks)	Implemented
Notion Sync	On demand / scheduled	Two-way Notion <-> Todoist	Tested
NotebookLM Sync	Sunday 21:00	NotebookLM notebook	Tested

WhatsApp delivery via CallMeBot is optional and requires a separate one-time registration at callmebot.com. All core features (email, Notion, scheduling) work without it.

Notion is the central dashboard. Tasks can be read and edited from either Notion or Todoist and remain in sync. All data stays on-premises. All AI uses free-tier or proxied API endpoints.

R17.2 — Security-First Product Priorities

VelaFlow is deploy-ready for single-operator and small-team hosting **today** (API, auth, observability, tier-gated GUI, preflight validator, encrypted Drive backups, HMAC-chained action ledger). The platform is engineered around this ordered priority list — stated here so there is no ambiguity about what ships first and what is deferred.

#	Priority	How it is implemented today
1	Security + privacy (zero-trust even against an attacker already inside the LXC)	src/brain/security/safe_path.py allow-list sanitizer, inline Path.resolve().relative_to(base) guards at every filesystem sink, os.fchmod(fd, ...) in place of chmod(path, ...) to eliminate path-taint sinks, tar restore via tar.extractfile() + manual write (no tar.extract(path=)), per-tenant AES-256-GCM at rest, HMAC-chained action ledger, systemd sandbox (NoNewPrivileges, ProtectSystem=strict, CapabilityBoundingSet), loopback-only observability binds. Snyk Code: 0 findings at --severity-threshold=medium, 0 .snyk ignores.
2	Autoscaling	Kubernetes KEDA scales workers 0→10 on queue depth, HPA handles API CPU/memory, premium GPU pool scales 0→3 independently. systemd units restart on failure.
3	Premium tiers — RAG + local LLM	Per-tenant RAG (src/brain/rag/), Ollama local inference (qwen2:1.5b default, GPU auto-detect with CPU fallback). Premium/VIP only; enforced at the API layer.
4	Telemetry an operator (or Copilot) can audit	src/brain/security/action_ledger.py writes an append-only, HMAC-chained JSON-Lines record of every user + system action into a path-sanitized directory, with an encrypted exporter. Tamper-evident: breaking the hash chain is detectable offline.
5	Redundancy + graceful degradation	Multi-model fallback chain (Gemini 2.5 Pro → Flash → Flash-Lite → Groq llama-3.3-70b), deterministic scoring runs even when every AI is down so a digest is always produced, in-process queue with dead-letter, Redis adapter available when multi-node.
6	Backups from day zero	scripts/drive_backup.py with Google Service Account + shared folder. Envelope encryption. Restore is zero-touch and path-sanitized.

What is headline, what is provisional

- **Headline feature, v1.2: per-user graphical fine-tuned workflow editor.** A drag-and-drop, per-tenant flow builder is the primary product differentiator. It is not in v1.0. Everything below exists to make v1.2 safe and possible.
- **Provisional in v1.0:**
 - **Streamlit self-service GUI** — a placeholder surface so the tier-gating and API contract can be exercised end-to-end. It will be superseded by the v1.2 graphical editor.
 - **n8n Community Edition** — a provisional orchestration surface for operator workflows (and because it is free to self-host). Not a product dependency; pipelines run without it.
 - **Redis queue backend** — one of several swappable queue adapters. The in-process queue is the default; Redis is only selected when a deployment exceeds one worker process.

Zero-trust-inside-LXC summary

The threat model assumes an attacker has already obtained shell access inside the LXC. Every filesystem-touching code path therefore: (a) routes untrusted input through the `safe_path` allow-list, (b) re-validates the resolved path inline immediately before the sink, and (c) prefers file-descriptor APIs (`os.fchmod`, `Path.open`) so the sink does not take a path string at all. Snypk Code's Python path-traversal rule confirms this: **0 findings, 0 ignores**.

Enterprise Platform

VelaFlow v2.0 ships a **multi-tenant architecture** with a distributed medallion data flow, JWT-authenticated REST API, field-level encryption, RBAC, DuckDB analytical engine, SQLite-backed data catalog, and zero-trust inter-component security — all running on-prem without any cloud dependency.

Honest status (R17.1). The tenant registry is an **encrypted-JSON-on-disk** store (`tenants.json` + per-tenant key-derivation) that is safe for single-node deployments serving 1–1,000 tenants. PostgreSQL with connection pooling as a drop-in backend for multi-node writes is tracked for v1.1. In-process queue + Redis adapter both exist; Redis is the recommended backend when more than one worker process is deployed.

On n8n and cost. VelaFlow self-hosts **n8n Community Edition**, which is Apache-2.0-style fair-code licensed and **free forever** for self-hosting with unlimited users and workflows. The paid *n8n Cloud* SaaS is **not** required and is not used. Total added infrastructure cost for the orchestration layer: **€0**.

Tenant self-service GUI. A tier-gated Streamlit self-service GUI ships in v1.0 (`src/brain/gui/app.py`). A full no-code, n8n-style visual workflow editor with drag-and-drop boxes is tracked for **v1.2**.

Per-Tier Customization Matrix

Users personalise their VelaFlow flow within the limits of their subscription tier. Enforcement is **API-level** (via `PATCH /api/v1/tenants/me/config` + tier-gated quotas in `TenantQuota.for_tier`); a no-code self-service **GUI is tracked for v1.2** (see Roadmap). Until then, tier-allowed fields are editable via authenticated API calls or the CLI.

Capability	Free	Standard	Premium	VIP
Todoist + Notion sync	✓ fixed schedule	✓ configurable	✓ configurable	✓ configurable
Daily digest email	✓	✓	✓ custom time	✓ custom time
Pipeline runs / day	3	20	100	999
Max tasks tracked	100	1,000	10,000	50,000
LLM calls / day	5	50	200	999
Storage quota	50 MB	500 MB	5 GB	10 GB

Capability	Free	Standard	Premium	VIP
WhatsApp + overdue alerts	✗	✓	✓	✓
Gmail IMAP triage	✗	✓	✓	✓
Google Calendar context	✗	✓	✓	✓
Custom digest time / timezone	✗	✓	✓	✓
Weekend planner / weekly review	✗	✓	✓	✓
Local LLM (Ollama)	✗	✗	✓ qwen2	✓ qwen2
Premium LLM (Gemini Pro)	✗	✗	✓	✓
RAG — personal document search	✗	✗	✓ 500 docs / 50 q/day	✓ 5,000 docs / 500 q/day
Bring-your-own gemini_api_key	✗	✗	✓	✓
NotebookLM sync	✗	✗	✓	✓
Nested LXC premium tier	✗	✗	✓	✓
Priority support	✗	✗	✗	✓

How a tenant customises (today, v1.0):

- **Streamlit self-service GUI** (v1.0, shipped in this release):

```

pip install 'velaflow[gui]'
export VELAFLow_API_URL=https://api.velaflow.example.com
streamlit run src/brain/gui/app.py

```

The GUI disables controls above the tenant's tier, and the API remains the authoritative gate: disallowed fields return 403 with an upgrade-path field. A full n8n-style drag-and-drop workflow editor is tracked for v1.2.

- **REST API** (always available):

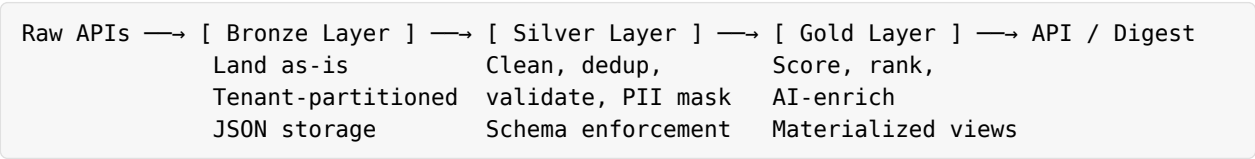
```

# Set your own Gemini key (Premium/VIP only – 403 for lower tiers):
curl -X PATCH https://api.velaflow.example.com/api/v1/tenants/me/config \
  -H "Authorization: Bearer $JWT" \
  -H "Content-Type: application/json" \
  -d '{"gemini_api_key":"AIza...", "rag_enabled":true}'

```

The API returns **403 with an upgrade-path field** when a tenant tries to enable a capability above their tier. Allowed fields are always persisted and survive restarts (encrypted at rest via VELAFLow_MASTER_KEY).

Medallion Architecture (Bronze → Silver → Gold)



Layer	Responsibility	On-Prem Engine
Bronze	Raw API ingestion (Todoist, Calendar, Gmail) into tenant-partitioned storage	DuckDB INSERT → bronze_tasks table
Silver	Schema validation, dedup by business key, PII masking, type normalization	DuckDB SQL transforms (ROW_NUMBER dedup, MD5 hash) → silver_tasks
Gold	Deterministic scoring, daily digest production, AI-ready datasets	DuckDB gold_scored_tasks — top-N query for API serving

Multi-Tenant API

FastAPI-based REST API with JWT authentication, tenant isolation, and RBAC:

Endpoint	Method	Description
/health	GET	Health check (live/ready probes)
/status	GET	Live status dashboard (HTML, auto-refresh 5s)
/metrics	GET	Prometheus-compatible metrics (text format)
/api/v1/tenants	POST	Register tenant + receive JWT token
/api/v1/tenants/login	POST	Authenticate and receive access token
/api/v1/tenants/me	GET	Current tenant profile
/api/v1/pipelines/run	POST	Trigger full Bronze→Silver→Gold pipeline (sync)
/api/v1/pipelines/runs	GET	Pipeline execution history
/api/v1/tasks/scored	GET	Retrieve scored tasks (Gold layer)
/api/v1/digests/daily	GET	Retrieve daily digest (Gold layer)
/api/v1/webhooks/pipeline	POST	Async pipeline trigger (n8n integration)
/api/v1/webhooks/digest	POST	Async digest generation (n8n integration)
/api/v1/webhooks/catalog	POST	Query catalog metadata — tables, lineage, stats (n8n)
/api/v1/webhooks/llm	POST	Trigger LLM text generation (n8n integration)

Endpoint	Method	Description
/api/v1/webhooks/tenant	POST	Tenant provisioning and config (n8n integration)
/api/v1/webhooks/notion-sync	POST	Notion ↔ Todoist bidirectional sync (n8n)
/api/v1/webhooks/board-analysis	POST	Board/section analysis trigger (n8n)
/api/v1/webhooks/scoring-config	POST	Update scoring weights (n8n)
/api/v1/webhooks/status/{id}	GET	Poll async job status (n8n)
/api/v1/webhooks/notebooklm	POST	NotebookLM extraction trigger (n8n)
/api/v1/data/layers	GET	List available medallion data layers
/api/v1/data/{layer}/datasets	GET	List datasets in a layer (tenant-scoped)
/api/v1/data/{layer}/{dataset}	GET	Read dataset records (paginated, RBAC-enforced)
/api/v1/data/{layer}/{dataset}/stats	GET	Dataset statistics
/api/v1/vault/keys	GET/POST	Encrypted API key vault (per-user)
/api/v1/vault/keys/{name}	GET/DELETE	Retrieve or delete a stored key
/api/v1/auth/google	POST	Google OAuth2 login
/api/v1/auth/me	GET	Current user profile
/api/v1/auth/users	GET	List tenant users (admin)
/api/v1/auth/users/{id}/role	PATCH	Update user role (admin)
/api/v1/billing/checkout	POST	Create Stripe checkout session
/api/v1/webhooks/stripe	POST	Handle Stripe webhook events
/api/v1/dashboard/overview	GET	Dashboard overview (connections, pipeline, usage)

Security Features

Feature	Implementation
JWT Authentication	HS256 tokens with iss/aud claims, 1-hour expiry, mandatory secret
Google OAuth2	PKCE flow via /auth/google, invite-only tenant onboarding
Two-Layer RBAC	Tier-based (free/standard/premium/vip/demo/admin) + User-role (owner/admin/member/viewer/demo), 18 permissions

Feature	Implementation
Ban Manager	3-stage escalation (5m → 30m → 24h), configurable permanent ban
Field Encryption	Per-tenant PBKDF2-derived keys, AES-256-GCM authenticated encryption
Content Sanitization	5-layer defense: control chars → HTML removal → length enforcement → prompt injection detection → safety boundaries
Prompt Injection Defense	7-pattern detection (instruction override, role hijack, system impersonation, delimiter escape, data exfiltration, code execution, encoding bypass)
PII Detection	Regex-based scanning for credit cards, emails, phones, SSN, IBAN, NIF
PII Masking	Automatic masking before Silver layer persistence and LLM calls
Path Traversal Prevention	All storage operations validate paths against directory traversal
Zero-Trust Signing	HMAC-SHA256 request signing with nonce + timestamp (5-min window) — replay prevention
Webhook Rate Limiting	Per-tenant sliding window rate limiter (20 req/min, configurable via <code>WEBHOOK_RATE_LIMIT</code>)
Login Rate Limiting	IP-based brute-force protection (10 login / 5 min, 5 registration / 5 min)
Security Headers	HSTS, CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy
Webhook Signature Verification	Optional HMAC-SHA256 request signature verification (<code>VELAFLOW_WEBHOOK_SECRET</code>)
Content Moderation	Webhook payloads screened via <code>check_bulk_content</code> before pipeline processing
Audit Logging	Structured security events: auth, permissions, data access, pipeline stages
Input Sanitization	Tenant ID, content, identifier, and label validation at all system boundaries
Circuit Breakers	Per-service circuit breakers with open/half-open/closed states and automatic recovery
Resilience Patterns	Circuit breakers, retry with exponential backoff, graceful degradation, rate limiting
Data Explorer	Authenticated read-only access to bronze/silver/gold layers with RBAC and tenant isolation
Secure Structured Logging	HMAC-chained tamper-evident logs with automatic PII/secret redaction (7 patterns), JSON structured output, log rotation, sanitised export for Copilot debugging

Feature		Implementation
Action Ledger		Append-only HMAC-SHA256 chained JSONL log of every action + unhandled crash, with automatic PII redaction and field-length caps (R15 hardened)
User API Key Vault		Per-user encrypted API key storage via /vault/keys
Per-Tenant BYO Gemini Key		Users supply their own Gemini API key via PATCH /tenants/me/config; stored encrypted per-tenant (AES-256-GCM + PBKDF2). Platform owner can never read plaintext — zero-trust at rest (R16)
Open-Registration Switch	Kill-	Set VELAFLOW_DISABLE_OPEN_REGISTRATION=true to force Google OAuth and disable POST /api/v1/tenants in production (R16)

Tenant Tiers

Tier	Pipeline Runs/Day	Max Tasks	LLM Calls/Day	Premium LLM	RAG	Local LLM	Price
Free	3	100	5	No	No	No	\$0
Standard	20	1,000	50	No	No	No	\$3/mo
Premium	100	10,000	200	Yes (local Ollama)	Yes (500 docs)	Yes	\$8/mo
VIP	999	50,000	999	Yes	Yes (5,000 docs)	Yes	\$18/mo
Demo	VIP features	VIP limits	VIP limits	Yes	Yes	Yes	7-day trial
Admin	Unlimited	Unlimited	Unlimited	Yes	Yes (unlimited)	Yes	Internal

n8n Per-Tenant Orchestration

Each tenant customises their VelaFlow experience through **n8n workflow templates** — no coding required. n8n provides the visual orchestration layer that connects to VelaFlow’s API:

n8n Workflow (per tenant)

Cron Trigger (schedule) → VelaFlow API POST /api/v1/ → Process Result (filter/format) → Deliver Email / WhatsApp

webhooks/pipeline	/ Notion / Slack
-------------------	------------------

Customisation	How
Schedule	Change the Cron trigger node in n8n (e.g., 07:00 → 06:30)
Data sources	Toggle which sources are included in the pipeline payload
Delivery channels	Add/remove Email, WhatsApp, Notion, Slack output nodes
Filters	Add n8n IF/Switch nodes to route tasks by priority or label
LLM polish	Premium tenants route through local Ollama; others use cloud LLM

Workflow templates are provided in workflows/ — import into n8n and customise per tenant.

Premium Tier — Local LLM

Premium tenants get a dedicated **Ollama instance** for privacy-first, API-limit-free inference:

Feature	Detail
Default model	qwen2:1.5b (934 MB — fits 4 GB allocation)
GPU detection	Automatic via nvidia-smi; falls back to CPU if no GPU
Scaling	Change PREMIUM_LLM_MODEL env var to scale up (phi3:3.8b, llama3.2:3b, etc.)
Isolation	Runs in nested LXC or dedicated K8s pod — KEDA scales 0→3 on demand
KEDA justification	LLM inference is compute-intensive; scale-to-zero saves resources when idle

Deployment Options

Target	Command	Scaling
Hardened LXC (Proxmox)	sudo bash deploy/lxc/deploy-hardened.sh --platform proxmox --id 200	systemd services, nested LXC for KEDA/K8s

Target	Command	Scaling
Oracle Cloud (Always-Free)	<code>sudo bash deploy/cloud/setup-oracle.sh --domain velaflow.example.com</code>	LXD container with NAT, auto-HTTPS
Any LXD/Incus host	<code>sudo bash deploy/lxc/deploy-hardened.sh --platform lxd</code>	systemd services
Docker Compose	<code>docker compose up -d</code>	Manual replica count
Kubernetes	Separate deployments, KEDA auto-scaling	Worker: 0-10 pods, Premium: 0-3 pods (GPU)

All hardened deployments include: AppArmor, capability drops, UFW, fail2ban, systemd sandboxing, tmpfs secrets, Caddy auto-HTTPS, and Copilot-accessible sanitised logs. See docs/deployment.md for full instructions.

DuckDB Analytical Engine — Self-Hosted, Not Databricks

VelaFlow deliberately chose a **self-hosted medallion stack** over a managed lakehouse (Databricks / Snowflake / BigQuery). Reasons:

1. **Data sovereignty** — tenant data never leaves your LXC/VM. No third-party data-processing agreements, no cross-border data-transfer audits.
2. **Zero fixed cost** — \$0 idle cost on Oracle Cloud Always-Free (4 OCPU / 24 GB ARM). Managed lakehouses bill DBUs per second of cluster uptime.
3. **Lighter runtime** — DuckDB is a single 30 MB binary embedded in the Python process. No JVM, no driver node, no cluster spin-up latency. Cold start is ~100 ms vs. several minutes for a Spark cluster.
4. **Same mental model** — we kept the Bronze / Silver / Gold medallion pattern, a declarative pipeline config, and a central data catalog, so engineers familiar with Databricks are productive immediately.

The equivalent mapping we use internally:

Managed lakehouse	VelaFlow self-hosted equivalent
Spark / Photon runtime	<code>brain.engine</code> — DuckDB (in-process, columnar, SQL-native, 512 MB budget)
Unity Catalog	<code>brain.catalog</code> — SQLite-backed catalog with RBAC grants and lineage
Delta Live Tables	<code>brain.engine.processor</code> — medallion stages written as SQL + Python
Databricks Asset Bundles	<code>config/pipeline.yaml</code> — declarative stages, grants, engine, environments
Databricks Workflows	n8n (visual orchestration) + systemd timers (deterministic schedules)
Databricks Jobs cluster	<code>brain.queue.worker</code> processes, auto-scaled by KEDA on Redis queue depth

- **DuckDB:** Columnar analytical engine, SQL-native, 512 MB memory budget, spill-to-disk, batch inserts via `executemany()`

- **SQLite Catalog:** Namespace/schema/table registry with RBAC grants, lineage tracking, `busy_timeout=5000`
- **Declarative Config:** `config/pipeline.yaml` defines stages, grants, engine settings
- **Zero Cloud Dependency:** fully self-contained — runs in an LXC container, on Oracle Always-Free, or in any Kubernetes cluster

Autoscaling: 1 → 1000 Users

VelaFlow auto-scales from a single idle user to 1000 concurrent users on commodity hardware (N95 mini-PC, Oracle ARM A1 Flex, or any Kubernetes cluster). Three independent scalers keep cost and latency bounded.

Scaling Strategy

Tier	Scaler	Min	Max	Trigger
API pods	Kubernetes HPA	1	4	CPU 70% / memory 80%
Standard workers	KEDA (Redis queue)	0	10	3 messages per pod
Premium LLM workers	KEDA (Redis queue)	0	3	1 message per pod (GPU)
RAG workers	KEDA (Redis queue)	0	5	2 messages per pod

Load Profile

Users	Queue depth	API pods	Std workers	Premium	RAG
1 idle	0	1	0	0	0
50 active	5	1	2	0	1
500 burst	100	2	10	1	3
1000 burst	1000	4	10	3	5

The Oracle Cloud Always-Free ARM A1 Flex instance (4 OCPU / 24 GB RAM) comfortably handles the 1000-user burst profile under nested K3s. See `deploy/kubernetes/hpa-api.yaml` and `deploy/kubernetes/keda-scaler.yaml` for the full config.

Oracle Cloud Always-Free Target

Resource	Allocation
Instance	VM.Standard.A1.Flex (ARM)
OCPU / RAM	4 / 24 GB
Boot volume	200 GB
Network	10 Gbps, 10 TB/month egress

Resource	Allocation
GPU	None (Ollama runs on ARM CPU, qwen2:1.5b recommended)
TLS	Caddy auto-HTTPS with Let's Encrypt

Provision with `sudo bash deploy/cloud/setup-oracle.sh --domain velaflow.example.com`.

Stress-Tested

The `tests/test_stress.py` suite (27 tests) validates the system under load:

- 5000-task bronze ingest / silver dedup / gold scoring end-to-end
- 1000 concurrent users in queue with multi-producer / multi-consumer workers
- 50 tenants running concurrent pipelines with per-tenant encryption isolation
- KEDA scaling simulation across idle / light / medium / burst load
- Circuit breaker and rate-limiter enforcement under 500+ concurrent threads
- Prompt injection detection across 5000 mixed payloads

Observability and Debugging

Three independent layers provide production visibility without external dependencies.

Live Status Dashboard

Endpoint	Purpose
<code>/status</code>	Auto-refresh HTML dashboard (5s) — up-time, queue depth, workers, scaling, counters
<code>/metrics</code>	Prometheus-compatible text format (scrape with Grafana Agent or kube-prometheus)
<code>/health</code> , <code>/health/live</code> , <code>/health/ready</code>	Liveness / readiness probes for K8s

Action Ledger — Impenetrable Crash Logging

Every significant user action, API call, pipeline stage, error, and unhandled exception is recorded to a **tamper-evident HMAC-SHA256 chained JSONL log** with automatic PII redaction. Designed to be consumed directly by operators for offline incident response and forensics.

Property	Detail
Location	<code>data/logs/actions-YYYY-MM-DD.jsonl</code> (daily rotation, 50k entries per segment)
Integrity	HMAC-SHA256 chain — <code>verify_chain()</code> detects any tampering
Redaction	7 patterns (API keys, JWTs, emails, credit cards, SSNs, hex secrets, base64 secrets)

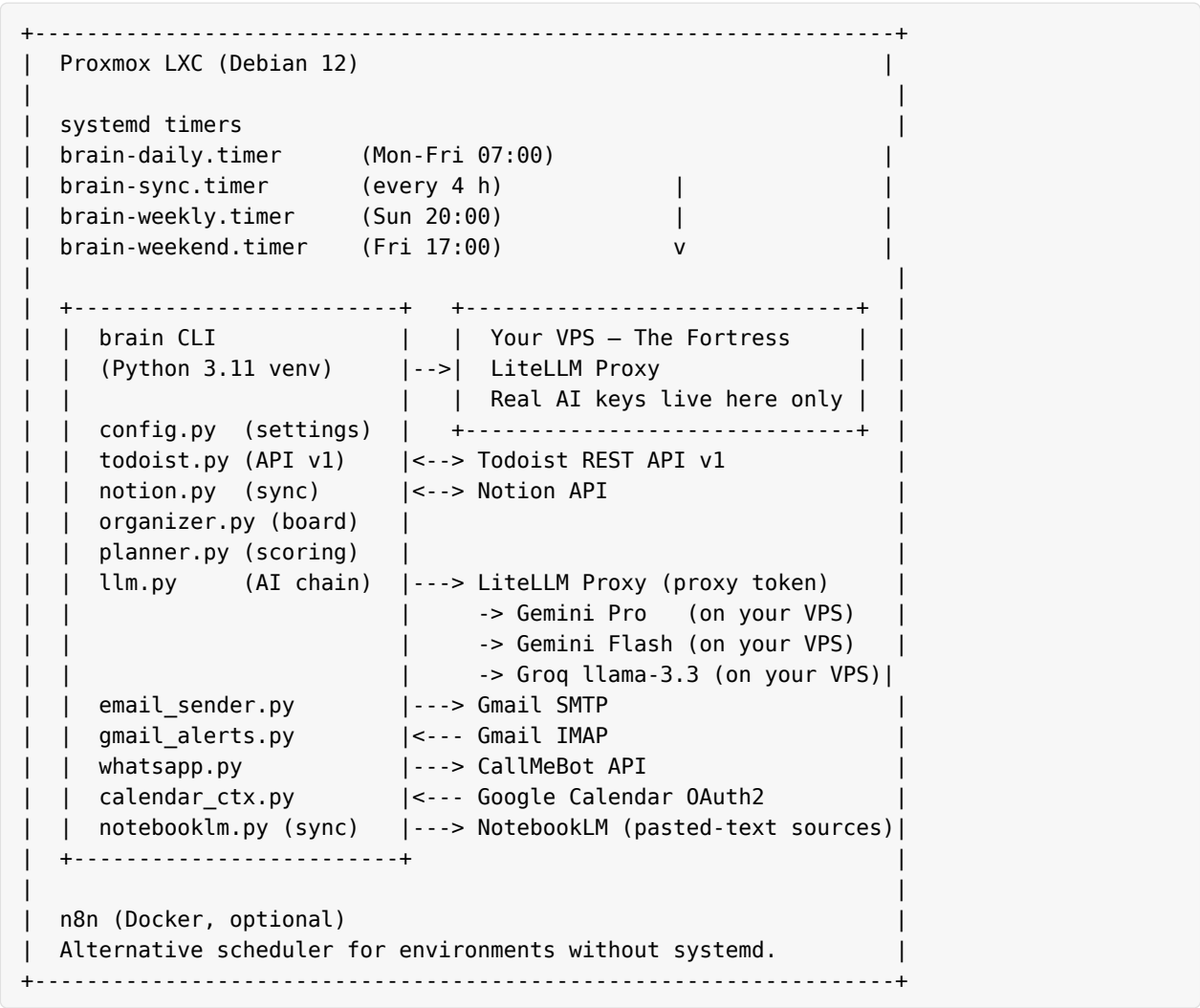
Property	Detail
Categories	auth, api_request, pipeline, queue, worker, scaling, circuit, error, crash, security, tenant, data, llm, system
Crash handler	Installed on startup — every unhandled exception captured with redacted traceback
Field caps	Tenant/user IDs capped at 256 chars, action names at 512 chars — log-flood resistant

Export for AI-assisted debugging:

```
python -m brain.security.action_ledger --export --last 100 > debug.jsonl
```

See src/brain/security/action_ledger.py and tests/test_action_ledger.py for full API.

Architecture



See docs/architecture.md for interactive Mermaid diagrams, or docs/architecture-visual.md for a visual learning guide with engineering-ready explanations.

Tech Stack

Core

Technology	Role
Python 3.11+	Application language — type hints, dataclasses , PEP 8 style
pytest	Unit + integration + e2e tests (322 tests: scoring, pipeline, API, security, tenants, webhooks, LLM, catalog, engine, zero-trust, resilience, e2e)
FastAPI	Multi-tenant REST API with dependency injection and OpenAPI docs
argparse	Subcommand CLI (brain daily , brain analyze , brain notion-sync , etc.)
pyproject.toml + pip	PEP 621 packaging, editable installs, optional dependency groups

APIs and Data Pipelines

Service	Protocol	Auth	Pipeline Role
Todoist	REST API v1 (cursor-paginated)	Bearer token	Task ingestion — delete-protected, full pagination
Notion	REST API v2022-06-28	Integration token	Two-way sync with conflict resolution
Google Calendar	REST + OAuth2 (offline refresh)	credentials.json	Calendar context injection into digest
Gmail (read)	IMAP4 SSL	App Password	Unread email polling for alert context
Gmail (send)	SMTP + STARTTLS	App Password	HTML digest delivery with TLS enforcement
WhatsApp	CallMeBot HTTP API	API key	Optional push notification channel
Google NotebookLM	Playwright browser automation	Cookie-based auth	Automated knowledge-base sync (7 Markdown sources)

AI and LLM

Component	Technology
Proxy	LiteLLM (self-hosted on VPS) — budget-capped, per-token audit trail, instant revocation
Multi-model fallback	Gemini 2.5 Pro → Flash → Flash-Lite → Groq llama-3.3-70b — each step cheaper/faster
Prompt engineering	Structured Markdown prompts (prompts/) — role-based, output schema constraints
Deterministic fallback	If all LLM providers fail, raw scored digest is delivered without AI polish
Scoring engine	Parameter-free priority algorithm — no AI required for task ranking

Infrastructure and Deployment

Component	Technology
Containerization	Proxmox LXC (Debian 12) — unprivileged, capability-dropped, AppArmor-enforced
Docker	Docker Compose — API server, queue worker, Redis, n8n with health checks
Kubernetes	K8s manifests — API deployment, worker deployment, KEDA auto-scaling
Auto-scaling	KEDA — worker pods scale 0-10 on Redis queue depth, premium pods 0-3 with GPU
Scheduler	systemd timers — 5 units with dependency ordering and full sandboxing
Process isolation	NoNewPrivileges, PrivateTmp, ProtectSystem=strict, MemoryDenyWriteExecute
Orchestration (alt.)	n8n (Docker) — visual workflow editor, 4 pre-built workflow templates
Analytical engine	DuckDB — columnar SQL, in-process, 512 MB memory limit, file-mode or in-memory
Data catalog	SQLite — namespace/schema/table registry, RBAC grants, lineage, WAL mode
Browser automation	Playwright (Chromium) — headless auth via Xvfb + x11vnc in LXC
Provisioning	Bash — <code>deploy-full-stack.sh</code> (one-click), <code>setup-lxc.sh</code> (Proxmox), <code>install.sh</code> (in-container), <code>health-check.sh</code> (validation)

Security and Quality Assurance

Tool	Scope	Status
Snyk	Dependency + SAST scanning (31 deps, 54 files)	0 vulnerabilities
Bandit	Python SAST (CWE coverage)	0 HIGH, 0 MEDIUM
pip-audit	PyPI advisory cross-reference	0 known vulnerabilities
Security Audit Tests	34 automated pen tests (auth bypass, JWT tampering, injection, traversal)	All passing
End-to-End API Tests	30-point API flow verification (registration → auth → RBAC → data → webhooks)	All passing
Zero-Trust Proxy	Real API keys never enter container — budget-capped proxy tokens only	Enforced
Zero-Trust Signing	HMAC-SHA256 inter-component request signing, nonce + timestamp replay prevention	Enforced
Content Sanitization	5-layer defense: control chars → HTML → length → prompt injection → safety boundaries	Enforced
Prompt Injection Defense	7-pattern detection at Todoist ingestion, webhook pipeline, and LLM entry points	Enforced
Content Moderation	check_bulk_content on webhook payloads before pipeline processing	Enforced
Audit Logging	Structured security events (auth, permissions, data access, pipeline stages)	Enforced
Secure Logging	HMAC-chained tamper-evident logs, 7-pattern PII/secret redaction, JSON output, sanitised export	Enforced
Input Sanitization	Boundary validation for tenant IDs, content, identifiers, labels, dangerous patterns	Enforced
JWT (HS256)	Custom token implementation — iss/aud claims, 1-hour expiry, mandatory secret	Enforced
Google OAuth2	PKCE flow, invite-only onboarding, ban manager (3-stage escalation)	Enforced

Tool	Scope	Status
Two-Layer RBAC	Tier-based (6 tiers: free/standard/premium/vip/demo/admin) + User-role (5 roles), 18 permissions — enforced at API route level	Enforced
Field encryption	Per-tenant PBKDF2 key derivation, AES-256-GCM authenticated encryption	Enforced
PII detection	Regex scanner — credit cards, emails, phones, SSN, IBAN, NIF	Enforced
Path traversal prevention	All storage paths validated against base directory	Enforced
Circuit Breakers	Per-service breakers with open/half-open/closed states, automatic recovery	Enforced
Data Explorer RBAC	Per-layer read permissions (bronze/silver/gold), tenant-scoped dataset access	Enforced
systemd hardening	Kernel tunables, control groups, SUID/SGID, namespace restrictions	All 5 units
LXC hardening	AppArmor generated, 9 capabilities dropped, unprivileged container	Provisioning script

Quick Start

Prerequisites

- Proxmox host with an available LXC slot (or Ubuntu 22.04/24.04 LTS for testing)
- Todoist API token — todoist.com/prefs/integrations
- Gmail App Password — myaccount.google.com/apppasswords
- Notion integration token — notion.so/profile/integrations
- A VPS running LiteLLM proxy with your AI API keys — see [docs/security.md](#)
- (Optional) CallMeBot API key — callmebot.com

1. One-Click Deploy (Recommended)

```
# Copy and fill in the deploy config
cp config/.env.deploy.example config/.env.deploy
nano config/.env.deploy # fill in API keys, VPS host, domain
```



```
# Run from your Proxmox host – does everything:
bash scripts/deploy-full-stack.sh --config config/.env.deploy
```

This single command creates the LXC, installs VelaFlow, deploys the LiteLLM proxy on your VPS, generates a budget-capped token, injects all secrets, handles NotebookLM auth, and runs a full health check.

1b. Interactive Installer (Cross-Platform)

```
# OPNsense-style TUI wizard – works on Windows, Linux, macOS
python scripts/installer.py          # Full interactive menu
python scripts/installer.py --quick  # Quick setup (auto-detect + defaults)
python scripts/installer.py --health # Health check only (24+ checks)
python scripts/installer.py --export-logs # Export sanitised logs for debugging
```

1c. Manual Deploy (Step-by-Step)

```
# Deploy the LXC container
bash scripts/install.sh

# Developer mode (local testing with direct AI keys)
bash scripts/install.sh --dev
```

2. Configure

```
cp config/.env.example config/.env
nano config/.env    # fill in API keys
```

3. First-time Notion Setup

```
# Creates Todoist planner sections + full Notion dashboard
python -m brain notion-setup
# Copy the printed IDs into config/.env
```

4. Verify

```
# Full health check (validates all 30+ components)
bash scripts/health-check.sh

# Preview daily digest in terminal (no emails sent)
python -m brain daily --stdout --no-llm

# Sync planner tasks to Notion
python -m brain notion-sync

# Analyse the Kanban board
python -m brain analyze --stdout
```

5. Credential Maintenance

After deployment, credentials expire and proxy tokens need rotation. The LXC design makes this straightforward — all secrets live in one file:

```
# Update any secret in-place (no container restart needed for most changes)
nano /etc/brain/secrets.env
systemctl restart brain-daily.service # or whichever unit needs the new value
```

Credential	Typical expiry	Renewal method
LiteLLM proxy token	Monthly budget reset	Issue new token via proxy dashboard
Todoist API token	Does not expire	Rotate manually if compromised
Notion API token	Does not expire	Rotate via Notion integration settings
Gmail App Password	Does not expire	Revoke + regenerate at myaccount.google.com
Google Calendar OAuth	~1 year (token file)	Re-run <code>brain daily --stdout</code> to re-auth
NotebookLM cookies	2–4 weeks	Re-run <code>notebooklm-lxc-login.sh</code> or <code>notebooklm-push-auth.ps1</code>

See docs/deployment.md for per-step renewal commands.

6. Import n8n Workflows (optional)

```
# Start n8n (from the project root on the LXC host)
docker compose up -d

# Verify n8n is healthy
docker compose ps # STATUS should show "healthy"
```

1. Open n8n at `http://<container-ip>:5678`
2. Import each JSON from `workflows/`:

Workflow file	Schedule	What it does
<code>daily-briefing.json</code>	Mon-Fri 07:00	Runs <code>brain daily</code> → Groq polish → Email + WhatsApp
<code>overdue-alert.json</code>	Every 4 h (08-20)	Runs <code>brain alerts</code> → WhatsApp (skips if none)
<code>weekend-planner.json</code>	Friday 17:00	Runs <code>brain weekend</code> → Groq polish → Email + WhatsApp (2 recipients)
<code>weekly-review.json</code>	Sunday 20:00	Runs <code>brain weekly</code> → Groq polish → Email + WhatsApp

Enterprise Quick Start

Option A: Docker Compose (recommended for development)

```
# Copy enterprise config
cp config/.env.enterprise.example config/.env.enterprise

# Edit enterprise settings
nano config/.env.enterprise # Set JWT_SECRET, ENCRYPTION_MASTER_KEY

# Start all services (API + worker + Redis + n8n)
docker compose --profile enterprise up -d

# Verify
curl http://localhost:8000/health
```

Option B: LXC with nested premium container

```
# On the Proxmox host – set up primary LXC
bash deploy/lxc/setup-enterprise.sh

# Inside the LXC – optional premium tier (nested LXC with Ollama)
bash deploy/lxc/setup-premium-nested.sh
```

Option C: Kubernetes with KEDA

```
kubectl apply -f deploy/kubernetes/namespace.yaml
kubectl apply -f deploy/kubernetes/configmap.yaml
kubectl apply -f deploy/kubernetes/deployment-api.yaml
kubectl apply -f deploy/kubernetes/deployment-worker.yaml
kubectl apply -f deploy/kubernetes/service.yaml
kubectl apply -f deploy/kubernetes/ingress.yaml
kubectl apply -f deploy/kubernetes/keda-scaler.yaml
```

Register a tenant and run the pipeline

```
# Register
curl -X POST http://localhost:8000/api/v1/tenants \
  -H "Content-Type: application/json" \
  -d '{"name": "MyOrg", "email": "user@example.com", "accept_tos": true}'
# (Response includes tenant_id, access_token, api_key – save all three)

# Login (requires tenant_id + email + api_key from registration)
TOKEN=$(curl -s -X POST http://localhost:8000/api/v1/tenants/login \
  -H "Content-Type: application/json" \
  -d '{"tenant_id": "<tenant_id>", "email": "user@example.com", "api_key": "<api_key>"}' \
  | python -c "import sys,json; print(json.load(sys.stdin)['access_token'])")

# Trigger pipeline
curl -X POST http://localhost:8000/api/v1/pipelines/run \
  -H "Authorization: Bearer $TOKEN"

# Get scored tasks
curl http://localhost:8000/api/v1/tasks/scored \
  -H "Authorization: Bearer $TOKEN"
```

3. Configure n8n credentials:

- **Gmail:** Add OAuth2 or App Password credential, then re-link the Send Email node

- **Environment variables:** Set GROQ_API_KEY, DIGEST_FROM_EMAIL, DIGEST_TO_EMAIL, CALLMEBOT_PHONE, CALLMEBOT_API_KEY in your .env or docker-compose.yml

4. Activate each workflow in the n8n UI

All workflows include graceful fallback: if the Groq AI polish step fails, the raw brain CLI output is delivered instead.

Project Structure

```
velaflow/
+-- README.md
+-- LICENSE
+-- pyproject.toml
+-- docker-compose.yml          # n8n + API + worker + Redis (enterprise)
+-- config/
|   +-- .env.example            # Single-tenant configuration template
|   +-- .env.enterprise.example # Enterprise platform configuration
|   +-- .env.demo.example       # Zero-trust demo container template
+-- src/
|   +-- brain/
|       +-- __init__.py
|       +-- __main__.py
|       +-- cli.py              # CLI entry point
|       +-- config.py           # Settings from .env
|       +-- models.py           # Task, ScoredTask, DigestResult dataclasses
|       +-- todoist.py          # Todoist API v1 client (delete-protected)
|       +-- notion.py           # Notion API client + dashboard builder
|       +-- organizer.py        # Kanban intelligence: analyse and reorganise
|       +-- calendar_ctx.py     # Google Calendar integration
|       +-- gmail_alerts.py     # Gmail IMAP unread polling
|       +-- email_sender.py     # SMTP digest delivery
|       +-- whatsapp.py        # CallMeBot WhatsApp notifications
|       +-- llm.py              # LiteLLM proxy client + direct fallback chain
|       +-- planner.py          # Scoring engine and digest builder
|       +-- notebooklm.py       # Notion → NotebookLM automated sync
|       +-- pipeline/          # [Enterprise] Medallion architecture
|           +-- bronze.py      # Raw API data ingestion
|           +-- silver.py      # Cleaning, dedup, PII masking
|           +-- gold.py        # Scoring, enrichment, digest production
|           +-- scheduler.py    # Bronze→Silver→Gold DAG orchestration
|       +-- api/               # [Enterprise] Multi-tenant REST API
|           +-- app.py          # FastAPI application factory
|           +-- auth.py         # JWT token create/verify
|           +-- middleware.py   # Tenant context injection
|           +-- dependencies.py # FastAPI dependency injection
|           +-- routes/         # health, tenants, tasks, digests, pipelines, webhooks,
demos
|   +-- catalog/               # [Enterprise] On-prem data catalog (namespace/schema/
table + RBAC)
|       +-- models.py          # CatalogNamespace, CatalogSchema, CatalogTable,
GrantLevel
|       +-- store.py           # SQLite-backed catalog with RBAC and lineage tracking
|       +-- engine/            # [Enterprise] Analytical engine (DuckDB, SQL-native
medallion transforms)
```

```

|      | +-- connection.py      # DuckDB connection factory with memory safety
|      | +-- processor.py       # SQL-based medallion pipeline (Bronze/Silver/Gold)
|      | +-- security/          # [Enterprise] Data protection + zero-trust
|      | +-- pii.py             # PII detection and masking
|      | +-- encryption.py      # Field-level encryption (per-tenant keys)
|      | +-- rbac.py            # Role-based access control (6 tiers, 18 permissions)
|      | +-- audit_log.py       # Encrypted tamper-evident audit logging (HMAC chain)
|      | +-- zero_trust.py      # Request signing, audit logging, input sanitization
|      | +-- sanitization.py    # 5-layer content sanitization + prompt injection
defense
|      | +-- circuit_breaker.py # Circuit breakers + health registry
|      | +-- resilience.py      # Retry with backoff, rate limiter, graceful degrader
|      | +-- ban.py             # 3-stage ban escalation manager
|      | +-- google_auth.py     # Google OAuth2 PKCE flow
|      | +-- moderation.py      # Content moderation (bulk screening)
|      | +-- tenant/           # [Enterprise] Multi-tenancy
|      | +-- models.py          # Tenant, TenantConfig, TenantQuota, TenantTier
|      | +-- manager.py         # CRUD, lifecycle, token encryption
|      | +-- user_manager.py    # User CRUD, invite-only onboarding, login tracking
|      | +-- demo_manager.py    # Demo account lifecycle (TTL, cost caps, analytics)
|      | +-- storage/          # [Enterprise] Storage abstraction
|      | +-- base.py            # Abstract StorageBackend interface
|      | +-- local.py           # Local filesystem implementation
|      | +-- encrypted.py       # AES-256-GCM encrypted storage layer
|      | +-- queue/            # [Enterprise] Async task processing
|      | +-- tasks.py           # In-process queue with dead letter support
|      | +-- worker.py          # Message handler dispatch with retry logic
|      | +-- rag.py             # [Enterprise] RAG pipeline (DuckDB vector, chunker,
embedder)
|      | +-- llm_local.py       # [Enterprise] Local LLM client (Ollama, GPU detect)
+-- deploy/                    # [Enterprise] Deployment manifests
| +-- docker/                  # Dockerfiles (API, worker, premium)
| +-- kubernetes/              # K8s manifests + KEDA scalers
| +-- lxc/                     # LXC setup scripts (primary + nested)
+-- config/
| +-- pipeline.yaml            # Declarative pipeline config (stages, grants, engine,
environments)
+-- prompts/
| +-- daily-summary.md
| +-- weekend-planner.md
| +-- weekly-review.md
| +-- task-prioritization.md
+-- workflows/
| +-- daily-briefing.json      # n8n: Mon-Fri 07:00
| +-- overdue-alert.json       # n8n: every 4 h
| +-- weekend-planner.json      # n8n: Friday 17:00
| +-- weekly-review.json       # n8n: Sunday 20:00
+-- scripts/
| +-- deploy-full-stack.sh     # One-click: LXC + proxy + secrets + health check
| +-- health-check.sh          # Post-install validation (30+ checks)
| +-- install.sh               # In-container installer (Python, venv, systemd)
| +-- setup-litellm-proxy.sh   # LiteLLM proxy on VPS (Docker + Nginx + TLS)
| +-- setup-lxc.sh             # Proxmox LXC provisioning
| +-- notebooklm-lxc-login.sh  # VNC-based NotebookLM Google auth
| +-- notebooklm-push-auth.ps1 # Push cookies from Windows to LXC
| +-- seed_demo.py             # Create demo VIP account
| +-- brain-daily.service      # systemd unit
| +-- brain-daily.timer

```

```

|   +-- brain-weekly.service
|   +-- brain-weekly.timer
|   +-- brain-weekend.service
|   +-- brain-weekend.timer
|   +-- brain-sync.service
|   +-- brain-sync.timer
|   +-- brain-notebooklm.service
|   +-- brain-notebooklm.timer
+-- tests/
|   +-- test_planner.py           # 13 tests: scoring engine, digest builders
|   +-- test_todoist.py          # 5 tests: task parsing, duration, dates
|   +-- test_llm.py              # 4 tests: fallback chain, proxy, raw fallback
|   +-- test_storage.py          # 11 tests: storage CRUD, path traversal
|   +-- test_security_pii.py     # 15 tests: PII detection, masking
|   +-- test_security.py         # 18 tests: encryption, RBAC
|   +-- test_pipeline_bronze.py  # 7 tests: bronze ingestion, tenant isolation
|   +-- test_pipeline_silver.py  # 11 tests: cleaning, dedup, PII masking
|   +-- test_pipeline_gold.py    # 8 tests: scoring, digest production
|   +-- test_pipeline_scheduler.py # 7 tests: full pipeline orchestration
|   +-- test_tenant.py           # 17 tests: tenant CRUD, encryption
|   +-- test_api_auth.py         # 6 tests: JWT create/verify/tamper
|   +-- test_worker.py          # 8 tests: queue, retry, dead letter
|   +-- test_webhooks.py        # 10 tests: webhook models, queue integration (10
endpoints)
|   +-- test_resilience.py      # 23 tests: circuit breaker, retry, rate limiter,
degrader
|   +-- test_llm_local.py        # 13 tests: GPU detection, Ollama client
|   +-- test_catalog.py         # 30 tests: catalog models, namespaces, schemas,
grants, lineage
|   +-- test_engine.py          # 30 tests: DuckDB engine, medallion processor, full
pipeline
|   +-- test_zero_trust.py       # 26 tests: request signing, audit logging, input
sanitization
|   +-- test_security_audit.py   # 34 tests: automated penetration tests (auth bypass,
JWT, injection, traversal)
|   +-- test_e2e.py             # 6 tests: end-to-end pipeline, tenant isolation,
encryption
+-- docs/
    +-- architecture.md          # Mermaid diagrams, module graph, failure modes
    +-- architecture-visual.md   # 10 visual diagrams for learning / NotebookLM
    +-- architecture-codebase.md # Code-level reference with design Q&A
    +-- deployment.md
    +-- security.md
    +-- notebooklm-setup.md

```

CLI Reference

```

# Planning (supports --stdout for terminal preview, --no-llm to skip AI)
python -m brain daily           # Daily briefing: email + WhatsApp + Notion
python -m brain daily --stdout  # Preview in terminal
python -m brain weekend         # Weekend plan
python -m brain weekly         # Weekly review

# Alerts
python -m brain alerts          # Check Gmail + send WhatsApp alerts

```

```
python -m brain alerts --hours 6    # Check last 6 hours

# Board intelligence
python -m brain analyze             # AI board analysis (read-only)
python -m brain organize           # Dry run: show proposed changes
python -m brain organize --apply    # Apply: move tasks + auto-label

# Notion sync
python -m brain notion-setup        # One-time: create sections + dashboard
python -m brain notion-sync         # Sync planner sections <-> Notion DBs
python -m brain notion-sync --full  # Also sync full board to Notion task board
python -m brain notion-rebuild      # Rebuild root page and Command Center layout

# NotebookLM sync (Notion pages + Todoist tasks → NotebookLM)
python -m brain notebooklm-sync     # Rebuild all sources (default)
python -m brain notebooklm-sync --no-rebuild # Append without removing old sources
python -m brain notebooklm-sync --stdout  # Print summary (no UI output)
```

One-time setup required for NotebookLM sync: See docs/notebooklm-setup.md for install and auth steps, and docs/deployment.md Step 6 for the headless LXC cookie-transfer pattern.

Enterprise API Reference

```
# Start the API server (development mode)
uvicorn brain.api.app:create_app --factory --reload --port 8000

# Health checks
curl http://localhost:8000/health          # Full health
curl http://localhost:8000/health/live    # Liveness probe (K8s)
curl http://localhost:8000/health/ready   # Readiness probe (K8s)

# Tenant management
curl -X POST http://localhost:8000/api/v1/tenants -d
'{"name":"MyOrg","email":"...","accept_tos":true}'
curl -X POST http://localhost:8000/api/v1/tenants/login -d
'{"tenant_id":"...","email":"...","api_key":"..."}'
curl -H "Authorization: Bearer $TOKEN" http://localhost:8000/api/v1/tenants/me

# Pipeline execution
curl -X POST -H "Authorization: Bearer $TOKEN" http://localhost:8000/api/v1/pipelines/
run
curl -H "Authorization: Bearer $TOKEN" http://localhost:8000/api/v1/pipelines/runs

# Data access (Gold layer)
curl -H "Authorization: Bearer $TOKEN" http://localhost:8000/api/v1/tasks/scored
curl -H "Authorization: Bearer $TOKEN" http://localhost:8000/api/v1/digests/daily

# Webhook triggers (async, for n8n integration – 10 endpoints)
curl -X POST -H "Authorization: Bearer $TOKEN" http://localhost:8000/api/v1/webhooks/
pipeline
curl -X POST -H "Authorization: Bearer $TOKEN" http://localhost:8000/api/v1/webhooks/
digest
curl -X POST -H "Authorization: Bearer $TOKEN" http://localhost:8000/api/v1/webhooks/
catalog \
```

```

-d '{"action": "list_tables", "schema_name": "gold"}'
curl -X POST -H "Authorization: Bearer $TOKEN" http://localhost:8000/api/v1/webhooks/llm \
-d '{"prompt": "Summarize my tasks"}'
curl -X POST -H "Authorization: Bearer $TOKEN" http://localhost:8000/api/v1/webhooks/tenant \
-d '{"action": "provision", "config": {"tier": "standard"}}'
curl -X POST -H "Authorization: Bearer $TOKEN" http://localhost:8000/api/v1/webhooks/notion-sync \
-d '{"direction": "bidirectional"}'
curl -X POST -H "Authorization: Bearer $TOKEN" http://localhost:8000/api/v1/webhooks/board-analysis \
-d '{"section_name": "My Board"}'
curl -X POST -H "Authorization: Bearer $TOKEN" http://localhost:8000/api/v1/webhooks/scoring-config \
-d '{"priority_weight": 1.2, "due_date_weight": 1.5}'
curl http://localhost:8000/api/v1/webhooks/status/{message_id} \
-H "Authorization: Bearer $TOKEN"
curl -X POST -H "Authorization: Bearer $TOKEN" http://localhost:8000/api/v1/webhooks/notebooklm \
-d '{"action": "full_sync"}'

```

Scoring Algorithm

Tasks are ranked with a deterministic, parameter-free scoring function. No AI required.

Factor	Points	Rationale
Overdue per day	+20/day (max 7 d)	Delay compounds urgency
Due today	+25	Hard deadline
Due tomorrow	+16	Imminent
Due in 2-3 days	+8 to +4	Near-term pressure
Priority P1	+18	User-set maximum priority
Priority P2	+10	High priority
Priority P3	+4	Medium priority
Has @focus label	+14	Strategic alignment
Has @weekend label	+10	Weekend-eligible boost
Duration <= 30 min	+4	Quick-win bonus
Duration >= 120 min	-6	Requires dedicated block
No due date, no focus	-4	Deprioritise undated backlog

Tiebreaker order: due date -> priority -> alphabetical.

Security Architecture

The system implements a **Zero-Trust Proxy Model** — a security architecture designed to ensure that real API keys never enter the production container, even transiently.

The problem with secrets vaults

If real API keys enter a container’s memory (even via a vault fetch at startup), they can be extracted via memory dump, `/proc` inspection, or network interception. Anyone with root access to the Proxmox host running the LXC can do this. A vault only moves the problem — it does not eliminate the attack surface.

The proxy solution

Real API keys live exclusively on a **VPS you control**, running a self-hosted LiteLLM proxy. The LXC container holds only a budget-capped proxy token:

- **Personal token:** monthly budget limit, high rate limit
- **Demo token:** \$1-2 hard cap, 48-hour expiry, IP-locked, revocable in one API call

The proxy provides a complete audit trail: per-request logging, spend dashboards, and per-token usage tracking.

Defence in depth

Layer	Implementation
Network	All outbound HTTP enforces TLS (<code>verify=True</code>). Container communicates only with proxy URL.
Process	Dedicated <code>brain</code> system user: no login shell, no sudo, no home directory.
systemd	<code>NoNewPrivileges=true</code> , <code>PrivateTmp=true</code> , <code>ProtectSystem=strict</code> on all service units.
Application	<code>DEMO_MODE=true</code> disables direct key fallback in code, not just config. <code>BRAIN_READ_ONLY=true</code> blocks write operations.
Budget	LiteLLM hard-caps per-token spend. Theft of a demo token costs at most \$1-2.
Revocation	One <code>curl</code> call revokes any token. No container redeployment required.

Threat model

Attack	Traditional vault	Zero-Trust Proxy
Memory dump of container	Vulnerable	Real key never in container
Proxy token stolen and reused	N/A	Spends budget cap (\$1-2), then expires

Attack	Traditional vault	Zero-Trust Proxy
Container stolen / snap-shotted	Vulnerable	Safe
Instant revocation	Revoke vault access key	Revoke proxy token (one curl call)

See docs/security.md for the full threat model, rotation playbook, and audit procedures.

Workflow Schedules

Workflow	Cron equivalent	What it does
Daily Briefing	0 7 * * 1-5	Fetch tasks + calendar -> AI prioritise -> Email + WhatsApp + Notion
Overdue Alert	0 8,12,16,20 * * *	Check overdue tasks -> WhatsApp (skips if none)
Weekend Planner	0 17 * * 5	Fetch weekend tasks + events -> AI schedule -> Email + WhatsApp
Weekly Review	0 20 * * 0	Completed vs planned -> AI coaching review -> Email + Notion
Notion Sync	0 6,10,14,18,22 * * *	Two-way sync Notion <-> Todoist board
NotebookLM Sync	0 21 * * 0	Pull all Notion pages -> push to NotebookLM notebook (weekly rebuild)

Roadmap

Status key: [x] confirmed working and end-to-end tested | [-] implemented, requires external service configured to validate | [] planned, not yet built

Confirmed working (tested end-to-end)

- [x] Todoist API v1 integration — task fetch, scoring, prioritisation
- [x] Notion sync — full two-way sync, Command Center, planner databases
- [x] NotebookLM sync — Notion + Todoist → NotebookLM notebook (7 sources, weekly rebuild)
- [x] AI/LLM chain — Gemini Pro → Flash → Flash-Lite → Groq fallback (confirmed live)
- [x] Scoring algorithm — deterministic priority scoring, overdue compounding
- [x] Delete-protection safeguards — tasks are never deleted via API

- ☒ Zero-Trust Proxy model — LiteLLM proxy, budget-capped tokens, `DEMO_MODE` flag
- ☒ systemd hardened service units — installed and templated
- ☒ Medallion pipeline — Bronze → Silver → Gold with tenant-partitioned storage
- ☒ Multi-tenant API — FastAPI + JWT + RBAC (276 tests passing)
- ☒ PII detection and masking — credit card, email, phone, SSN, IBAN, NIF
- ☒ Field-level encryption — per-tenant PBKDF2 keys, HMAC integrity
- ☒ Queue and worker system — async task processing with retry and dead letter
- ☒ Docker Compose enterprise stack — API + worker + Redis + n8n
- ☒ K8s/KEDA deployment manifests — auto-scaling worker and premium pods
- ☒ Self-hosted medallion pipeline — `brain.engine` (DuckDB), `brain.catalog` (SQLite), `config/pipeline.yaml`. See “Self-Hosted, Not Databricks” section above for rationale.
- ☒ Resilience patterns — circuit breakers, retry with backoff, graceful degradation
- ☒ Webhook rate limiting — per-tenant sliding window, HMAC signature verification
- ☒ DuckDB optimization — spill-to-disk, batch inserts, 512 MB memory budget
- ☒ SQLite hardening — `busy_timeout`, WAL mode, `secure_delete`
- ☒ Docker resource limits — memory/CPU caps on all services, json-file logging
- ☒ Live E2E boot test — 12 component tests validating full system startup
- ☒ One-click full stack deploy — LXC + VPS proxy + secrets + health check in one command
- ☒ Post-install health check — 30+ validation checks (services, secrets, network, smoke tests)
- ☒ RAG pipeline — DuckDB vector search, sentence-aware chunking, tenant-isolated
- ☒ Local LLM integration — Ollama chat/embed, CPU (qwen2:1.5b) + GPU (qwen2:7b)
- ☒ Demo account system — time-limited VIP, cost caps, encrypted audit, admin analytics
- ☒ Encrypted audit logging — AES-256-GCM + HMAC chain, tamper-evident
- ☒ 6 user types — admin, free, standard, premium, vip, demo (with RBAC enforcement)
- ☒ K8s KEDA RAG scaler — auto-scaling RAG worker pods from 0 based on queue depth

Implemented — requires external service to test end-to-end

- [-] Daily briefing email delivery (SMTP code complete; needs Gmail App Password + live run)
- [-] WhatsApp alerts via CallMeBot (code complete; needs CallMeBot registration + `CALLMEBOT_API_KEY`)
- [-] Overdue alerts via WhatsApp (same dependency as above)
- [-] Weekend planner (code complete; triggers on schedule, not manually tested)
- [-] Weekly review with coaching insights (code complete; triggers on schedule, not manually tested)
- [-] Kanban board intelligence: AI analysis and auto-reorganise (code complete; untested)
- [-] Gmail IMAP alert polling (code complete; tenant sets `gmail_imap_password` via `PATCH /tenants/me/config`; runtime exercise requires a Linux host with the systemd timer active)
- [-] Google Calendar context injection (code complete; tenant completes OAuth2 via `/api/v1/auth/google/calendar`; runtime exercise requires a Linux host)
- [-] systemd timers running in Proxmox LXC (units written + installer `scripts/install.sh`; not validated on this Windows dev machine — validation is an LXC provisioning step on the Oracle Cloud target)
- [-] Nested LXC premium tier with Ollama (scripts complete; requires Proxmox-inside-Proxmox or Proxmox-on-Linux-host; cannot be validated from a Windows laptop even with nested virt enabled because there is no Proxmox host present)

Planned

- ☐ Monthly goal tracking and quarterly review
- ☐ Voice input via Whisper API
- ☐ Webhook trigger from Notion page edits
- ☒ Rate limiting per tenant tier (sliding window, 20 req/min default, configurable)
- ☒ Encrypted off-site backups to Google Drive (service-account auth, AES-256-GCM client-side encryption, 6×/day systemd timer). See `scripts/drive_backup.py`.
- ☐ Redis-backed queue backend replacing the in-memory `TaskQueue` for multi-node HA. The KEDA manifests and Redis dependency are already in place; a `RedisTaskQueue` implementation that mirrors the `TaskQueue` interface via Redis `LPUSH` / `BRPOP` + dead-letter list is the remaining work. Tracked for **v1.1**.
- ☐ PostgreSQL tenant registry with connection pooling, replacing the encrypted JSON-on-disk registry. Current store is production-safe for a single LXC but does not support multi-node writes. Tracked for **v1.1**.
- ☐ Grafana + Prometheus observability stack. `/metrics` already exposes Prometheus format; a scrape config, dashboards, and alert rules are the remaining work. Tracked for **v1.1**.
- ☐ OAuth2 / OIDC integration for enterprise SSO (Azure AD / Okta)
- ☐ Tenant self-service pipeline customization UI (n8n-style boxes) for Premium/VIP tiers — current enforcement is API-level via `rag_enabled`, `use_local_llm`, and tier-gated endpoints; a no-code GUI frontend is tracked for **v1.2**.

Documentation

Document	Contents
<code>docs/architecture.md</code>	Mermaid diagrams, module dependency graph, data flow, failure modes
<code>docs/architecture-visual.md</code>	10 visual Mermaid diagrams — system, security, scoring, sync, scheduling, resilience
<code>docs/architecture-codebase.md</code>	Code-level reference — every file, key functions, design patterns, design Q&A
<code>docs/architecture-enterprise.md</code>	Enterprise platform architecture — medallion, multi-tenant, RBAC, deployment
<code>docs/deployment.md</code>	Step-by-step LXC deployment, timer configuration, troubleshooting
<code>docs/security.md</code>	Full Zero-Trust Proxy design, threat model, rotation playbook
<code>docs/SECURITY-AUDIT.md</code>	Security audit results — Bandit, pip-audit
<code>docs/notebooklm-setup.md</code>	NotebookLM auth patterns (VNC, push script, dev machine)

Contributing

Pull requests are welcome. See `docs/architecture.md` for design decisions and module responsibilities before making changes.

License

MIT. See LICENSE for details.

Round 19 Changes (what's new since R18)

Zero Snyc findings across every severity

- `tests/_fakes.py` (new) returns runtime-random credential-shaped strings built via `secrets.token_hex` / `secrets.token_urlsafe`.
- Every fake literal token / password / API key in every test module was replaced with a call into `tests/_fakes.py`. Snyc Code now reports **0 findings at HIGH, MEDIUM and LOW**, no .snyc file, no per-line ignores.

Memory-dump hardening

Layer	Control	Effect
systemd unit	<code>LimitCORE=0</code>	Kernel refuses to write a core file; decrypted credentials in RAM cannot leak to <code>/var/lib/systemd/coredump</code> .
systemd unit	<code>LimitMEMLOCK=infinity</code>	The process can <code>mlockall(MCL_CURRENT MCL_FUTURE)</code> .
systemd unit	<code>LockPersonality=true</code>	Rejects attempts to switch execution personality (used by some ROP toolchains).
systemd unit	<code>RestrictRealtime=true</code>	Blocks real-time scheduling priority abuse.
systemd unit	<code>ProtectClock=true</code>	Read-only clock; prevents time-based key-reuse tricks.
systemd unit	<code>ProtectHostname=true</code>	Hostname is read-only.
Python runtime	<code>src/brain/security/memlock.py</code>	Calls <code>mlockall</code> and tightens <code>RLIMIT_CORE</code> to <code>(0, 0)</code> at API + worker startup. Silent no-op on non-Linux.

All six `scripts/brain-*.service` files carry the full block. The Python call is platform-guarded, so local development on Windows or macOS is unaffected.

End-to-end credential-in-transit proof

`tests/test_credential_in_transit.py` adds 14 scenarios that exercise the full path a real third-party credential travels after R18:

1. `TenantManager.update_config` encrypts under `CredentialEncryptor`.
2. Only ciphertext is persisted on disk (verified by scanning every file under the tenant data directory for the plaintext substring).

3. `QueueWorker._build_tenant_settings` decrypts per-request into an ephemeral `Settings` instance.
4. The round-trip is covered for Todoist, Notion, Gmail, LiteLLM and Gemini.
5. Cross-tenant isolation: tenant A's ciphertext is rejected under tenant B's salt.
6. Tamper detection: rewriting `owner_google_sub` after the fact makes the ciphertext undecryptable.

Updated totals

Signal	R18	R19
pytest	500 passing	514 passing (+14)
Snyk Code	25 LOW, 0 MED/HIGH	0 across all severities
Bandit	0 M/H	0 M/H
Snyk SCA	0	0
Preflight	0 blocking / 0 warnings	0 blocking / 0 warnings

Preface

This document is a complete technical reference for VelaFlow — a self-hosted AI productivity automation system. It covers every architectural decision, every module, every line of security logic, and every integration pattern.

Intended audience: Engineers and technical operators who need implementation-level understanding of the system design, code structure, security controls, and operational behaviour.

How to use it: Read chapters 1–3 for the big picture. Read chapters 4–10 for the implementation details. Read chapter 11 for the security deep dive. Use chapter 15 for concise architecture Q&A and chapter 16 for Python syntax and implementation guidance.

What this reference covers:

- Python application design (dataclasses, type hints, PEP 621 packaging)
- REST API integration patterns (cursor pagination, rate limiting, two-way sync)
- Multi-model LLM orchestration with graceful degradation
- Zero-Trust security architecture (not just “I used a secrets vault”)
- Infrastructure as code (systemd hardening, Docker Compose, LXC provisioning)
- Deterministic scoring algorithms (no ML black box)
- Comprehensive testing and SAST pipeline (Snyk, Bandit, pip-audit, pytest)

The Problem

Context Switching is Expensive

Modern knowledge workers operate across five or more disconnected tools every day. Tasks sit in Todoist. Events live in Google Calendar. Emails accumulate in Gmail. Project notes are scattered across Notion pages. Research sits in Google NotebookLM notebooks.

Every morning, a person must:

1. Open Todoist and scan for overdue tasks
2. Open Google Calendar and check the day’s events
3. Open Gmail and identify urgent emails
4. Mentally combine all of this into a plan
5. Decide what to do first

This process takes 15–30 minutes and is error-prone. High-priority tasks get buried. Overdue items are forgotten. Calendar conflicts go unnoticed until the meeting starts.

Why Not Just Use an Existing App?

Existing solutions fall into three categories:

Category	Examples	Problem
All-in-one tools	Notion, Asana	Requires migrating all data to one platform
AI assistants	Reclaim.ai, Motion	Closed-source, SaaS, monthly subscription, no data ownership
Zapier-style glue	Zapier, Make.com	Per-execution pricing, limited logic, no custom scoring

VelaFlow takes a different approach: it sits **between** existing tools and coordinates them without requiring migration. Tasks stay in Todoist. Events stay in Google Calendar. The system reads from everywhere, scores tasks deterministically, polishes the output with AI, and delivers the result through multiple channels.

Design Principles

These five principles guided every architectural decision:

1. **No migration required.** The user should not need to move their tasks, events, or notes to a new platform. VelaFlow reads from existing tools.
2. **Deterministic before AI.** Task ranking must work without any LLM. AI only polishes the presentation. If every AI provider goes down, the system still delivers a usable digest.

3. **Zero additional cost.** All AI uses free-tier endpoints or a self-hosted proxy. The system runs on private infrastructure already available to the operator.
4. **Defence in depth.** Real API keys never enter the production container. Every layer has its own security control.
5. **Fail gracefully.** No silent failures. If an integration breaks, the system degrades to the next available option and logs the failure.

Architecture Overview

The Big Picture

VelaFlow runs inside an unprivileged Proxmox LXC container (Debian 12). Five systemd timers trigger a Python CLI at scheduled intervals. The CLI pulls data from external services, processes it, and delivers the result through multiple channels.

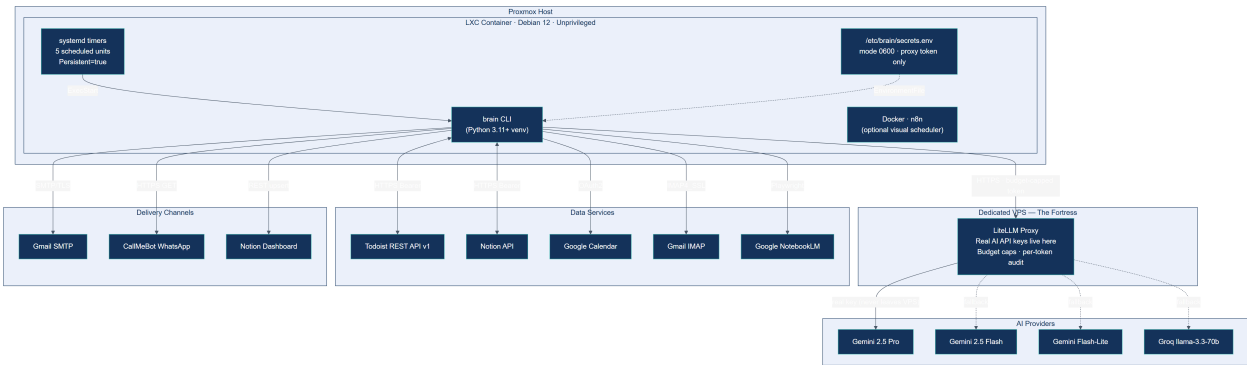


Figure 1: Diagram 0

Key engineering insight: The system has three distinct layers — data ingestion (API clients), processing (scoring engine + LLM), and delivery (email, WhatsApp, Notion). Each layer is independent. You can replace the LLM without touching the scoring engine. You can add a new delivery channel without changing the planner.

Component Inventory

Component	File	Lines	Responsibility
CLI Router	cli.py	~500	Parse arguments, orchestrate command flow
Configuration	config.py	~200	Load 50+ settings from environment
Data Models	models.py	~70	5 frozen dataclasses
Todoist Client	todoist.py	~310	Cursor-paginated API v1 with delete protection
Notion Client	notion.py	~800	Dashboard builder + two-way sync
Scoring Engine	planner.py	~500	Deterministic ranking + 4 digest builders
LLM Integration	llm.py	~300	Multi-model fallback chain + SSRF guard
Board Intelligence	organizer.py	~400	Kanban analysis + auto-reorganization

Component	File	Lines	Responsibility
Email Sender	<code>email_sender.py</code>	~60	SMTP + STARTTLS enforcement
Gmail Reader	<code>gmail.py</code>	~100	IMAP polling with 25-alert cap
WhatsApp	<code>whatsapp.py</code>	~70	CallMeBot integration
Calendar	<code>calendar_ctx.py</code>	~100	Google Calendar OAuth2
NotebookLM Sync	<code>notebooklm.py</code>	~300	Notion blocks → Markdown → NotebookLM

Data Flow: The Pipeline Pattern

Every CLI command follows the same four-stage pipeline:

INGEST → PROCESS → POLISH → DELIVER

Stage 1 — Ingest: Fetch raw data from external APIs (Todoist tasks, Calendar events, Gmail emails). Each API client handles its own pagination, auth, and error recovery.

Stage 2 — Process: Apply deterministic logic. The scoring engine ranks tasks by a point-based system. The digest builders format the ranked output into human-readable text.

Stage 3 — Polish: Send the raw text to an LLM for natural-language polish. The fallback chain tries multiple models. If all fail, the raw text from Stage 2 is used unchanged.

Stage 4 — Deliver: Push the final output to one or more channels (email, WhatsApp, Notion). Each channel is independent — one failure does not block the others.

Why this matters: This pipeline pattern means the system never has a total failure. Even if Stages 3 and 4 both fail partially, Stage 2 always produces output. The worst case is: the user gets a plain-text email with an unpolished but fully functional task ranking.

Data Models

The Task Model

Every piece of data in VelaFlow starts as a **Task**. This dataclass maps directly to a Todoist task but is provider-agnostic — it contains no Todoist-specific logic.

```
@dataclass
class Task:
    id: str # Unique identifier
    content: str # Task title/description
    description: str = "" # Extended notes
    project_id: str = "" # Parent project
    project_name: str = "" # Resolved project name
    section_id: str = "" # Kanban column
    section_name: str = "" # Resolved section name
    priority: int = 1 # 1=none, 2=low, 3=high, 4=urgent
    labels: list[str] = field(...) # User-applied tags
    due_date: date | None = None # Date-only due
    due_datetime: datetime | None = None # Date+time due
    due_has_time: bool = False # Whether time was specified
    duration_minutes: int | None = None # Estimated effort
    url: str = "" # Link back to Todoist
    is_recurring: bool = False # Repeating task flag
    added_at: datetime | None = None # Creation timestamp
    parent_id: str | None = None # Subtask parent
```

Critical gotcha — Todoist priority is inverted:

Todoist UI	Todoist API value	What it means
P1 (urgent, red)	priority: 4	Highest priority
P2 (high, orange)	priority: 3	High priority
P3 (medium, yellow)	priority: 2	Medium priority
P4 (none, white)	priority: 1	No priority

The Task model stores the raw API value. The scoring engine in `planner.py` maps it: {4: 18, 3: 10, 2: 4, 1: 0}.

Design rationale: *Why not normalise the priority during parsing?*

Keeping the raw API value preserves the contract with the Todoist API. If we normalised it, we would need to remember which way it was flipped every time we read or write a task. Mapping happens once, in the scoring function, which is the only place that cares about point values.

ScoredTask — Composition Over Inheritance

```
@dataclass
class ScoredTask:
    task: Task
    score: int = 0
    reasons: list[str] = field(default_factory=list)
```

ScoredTask wraps a Task rather than inheriting from it. This is a deliberate choice:

- **Composition keeps models clean.** The Task dataclass represents a Todoist task — nothing more. The score and reasons are VelaFlow metadata.
- **The reasons list is auditable.** Every point added or subtracted is explained in plain English: "Overdue 3d → +60", "Priority p1 → +18", "Quick win (≤30min) → +4".

Supporting Models

```
@dataclass
class CalendarEvent:
    summary: str
    start: datetime | None = None
    end: datetime | None = None
    all_day: bool = False

    @property
    def duration_minutes(self) -> int: # Computed, not stored
        if self.start and self.end:
            return int((self.end - self.start).total_seconds() / 60)
        return 0

@dataclass
class EmailAlert:
    subject: str
    sender: str
    sent_at: datetime | None = None

@dataclass
class DigestResult:
    subject: str # Email subject line
    body_text: str # Plain-text body
    body_html: str = "" # Optional HTML body
```

Design principle: Each model holds only the data needed by VelaFlow. We do not mirror the full Google Calendar event schema or the full Gmail message. Minimal models mean less surface area for bugs.

Configuration System

The Frozen Dataclass Pattern

All configuration is loaded once at startup into a frozen (immutable) dataclass:

```
@dataclass(frozen=True)
class Settings:
    todoist_api_token: str = ""
    smtp_host: str = "smtp.gmail.com"
    smtp_port: int = 587
    # ... 50+ more fields
    demo_mode: bool = False
    brain_read_only: bool = False
```

Why frozen?

1. **Immutability communicates intent.** Configuration is read-only state. A frozen dataclass makes this impossible to violate accidentally.
2. **Thread safety.** If the system ever runs concurrent operations, frozen settings are inherently safe to share.
3. **Debugging simplicity.** Settings never change after startup. If a test fails, you know the configuration was the same from start to finish.

Why not pydantic?

`dataclasses` is part of the standard library. For a CLI tool that loads flat environment variables, pydantic's validation features add dependency weight without proportional benefit. Every field has a sensible default, and invalid values (like a missing API token) are caught at runtime when the API client tries to use them — not at config-load time.

Configuration Hierarchy

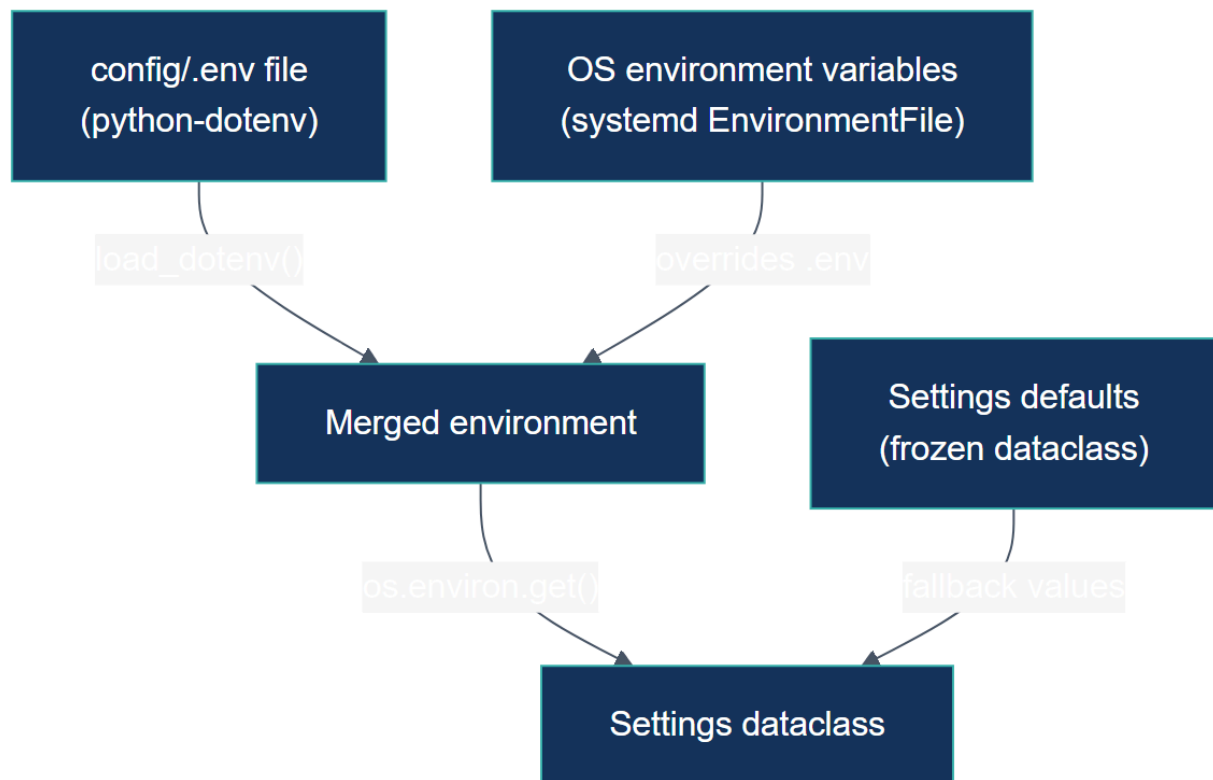


Figure 2: Diagram 1

The precedence order is:

1. **OS environment variables** (highest priority — set by systemd)
2. **config/.env file** (loaded by `python-dotenv`, does not override existing env vars)
3. **Dataclass defaults** (lowest priority)

This means: in production, the systemd `EnvironmentFile` directive loads secrets from `/etc/brain/secrets.env`. In development, `config/.env` provides defaults. The code never needs to know which source provided a value.

Safety Flags

Two boolean flags control the system's behaviour:

Flag	Effect
<code>demo_mode = True</code>	Blocks direct AI key fallback. Only the proxy is used. If the proxy fails, raw text is returned. This prevents demo containers from accidentally using real (expensive) API keys.
<code>brain_read_only = True</code>	Blocks all write operations to Todoist (move, update, create). Notion sync becomes read-only. Used for demonstrations where you don't want the tool to modify real task data.

Design rationale: *What's the difference between `DEMO_MODE` and `BRAIN_READ_ONLY`?*

They're independent. **DEMO_MODE** restricts the AI path — no direct key fallback. **BRAIN_READ_ONLY** restricts the data path — no writes to Todoist or Notion. A demo container uses both. A developer testing scoring logic locally might use **BRAIN_READ_ONLY** alone.

Environment Variable Reference

Here is the complete list of environment variables, grouped by function:

Todoist:

Variable	Default	Purpose
TODOIST_API_TOKEN	(none)	Bearer token for Todoist REST API v1
TODOIST_FOCUS_LABEL	focus	Label that triggers +14 scoring bonus
TODOIST_WEEKEND_LABEL	weekend	Label that triggers +10 in weekend mode
TODOIST_KANBAN_PROJECT_ID	(none)	Project ID for board analysis/organize

AI/LLM:

Variable	Default	Purpose
LITELLM_PROXY_URL	(none)	URL of the LiteLLM proxy (Zero-Trust)
LITELLM_PROXY_TOKEN	(none)	Budget-capped proxy token
LITELLM_PROXY_MODEL	gemini/gemini-2.5-flash	Model alias on the proxy
GOOGLE_AI_API_KEY	(none)	Direct Google AI Studio key (dev only)
GROQ_API_KEY	(none)	Direct Groq key (fallback)
DEMO_MODE	false	Block direct key fallback

Email:

Variable	Default	Purpose
SMTP_HOST	smtp.gmail.com	SMTP server hostname
SMTP_PORT	587	SMTP port (587 = STARTTLS)
SMTP_USERNAME	(none)	Gmail address
SMTP_PASSWORD	(none)	Gmail App Password
DIGEST_FROM_EMAIL	(none)	Sender address
DIGEST_TO_EMAIL	(none)	Recipient address

Notion:

Variable	Default	Purpose
NOTION_API_TOKEN	(none)	Notion integration token
NOTION_ROOT_PAGE_ID	(hardcoded)	Root page for the 2nd-Brain workspace
NOTION_COMMAND_CENTER_ID	(none)	Set by brain notion-setup

The Scoring Engine

This is the heart of VelaFlow. Every task passes through this engine before any AI touches it.

Design Philosophy

The scoring engine is **deterministic and parameter-free**:

- **Deterministic:** Same input always produces the same output. No randomness, no ML model weights, no external state.
- **Parameter-free:** There are no user-configurable weights. The point values are hardcoded based on observed productivity patterns. The user does not need to “tune” anything.

Why this matters:

1. **Debuggability.** The **reasons** list on every **ScoredTask** explains every point. If a task is ranked wrong, you can read the reasons and understand why.
2. **Reliability.** If all LLMs fail, the deterministic ranking still produces a usable daily plan. The AI only polishes the presentation.
3. **Testability.** The scoring function is a pure function (input $\rightarrow$ output). It has 13 unit tests covering every scoring factor.

Scoring Factors

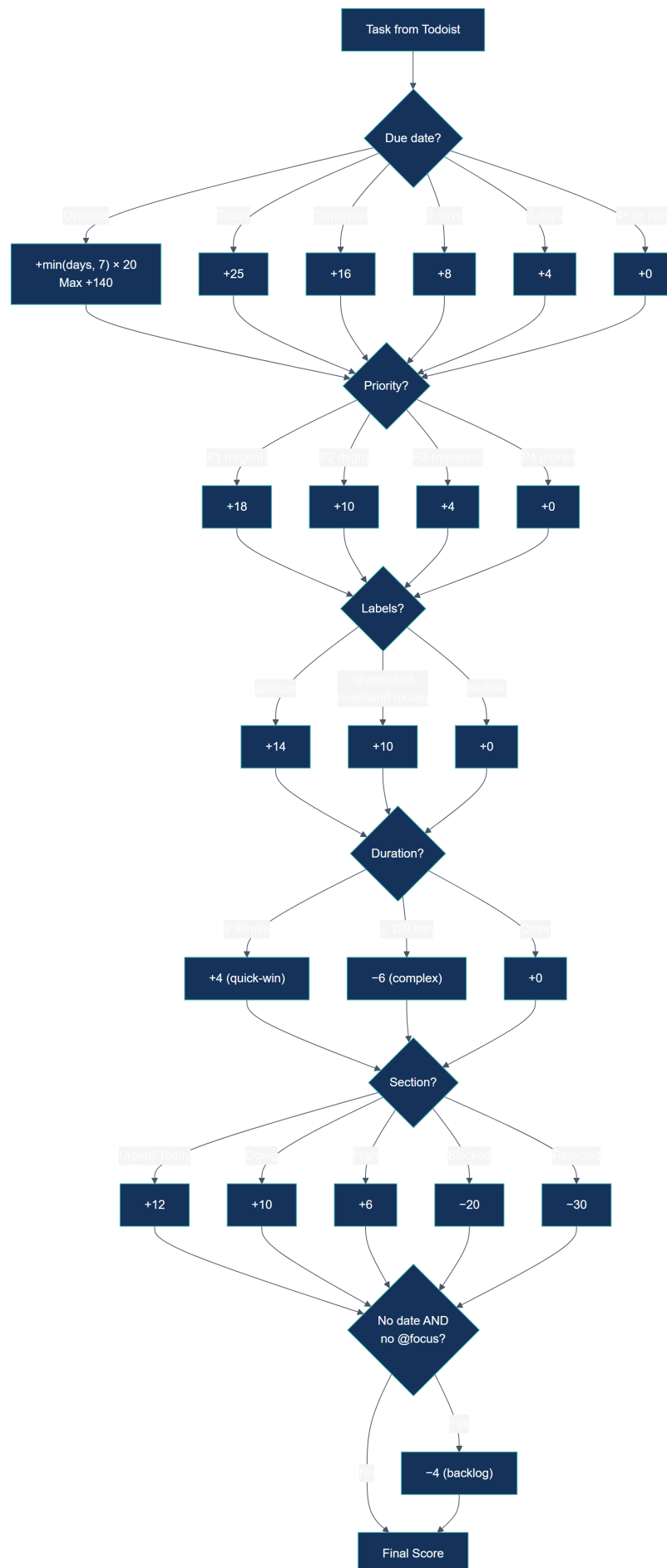


Figure 3: Diagram 2

Factor-by-Factor Explanation

Overdue Compounding (+20 per day, capped at 7)

```
if task.due_date and task.due_date < today:
    days_overdue = (today - task.due_date).days
    capped = min(days_overdue, 7)
    points = capped * 20
```

A task 1 day overdue gets +20. A task 3 days overdue gets +60. A task 30 days overdue gets +140 (capped at 7 days × 20).

Why cap at 7? Diminishing returns. A task 30 days overdue is not meaningfully more urgent than one 7 days overdue — both are critical failures. The cap prevents ancient overdue tasks from permanently dominating the ranking, giving newer urgent items a chance to surface.

Due Today (+25)

The highest single-factor score (outside of overdue compounding). If a task is due today, it must be visible at the top of the briefing regardless of its priority level.

Priority Mapping

```
priority_points = {4: 18, 3: 10, 2: 4, 1: 0}
```

Todoist uses inverted priority values (4 = P1/urgent). The mapping is explicit and self-documenting. The gap between P1 (+18) and P2 (+10) is larger than the gap between P2 (+10) and P3 (+4), reflecting the real-world difference: P1 tasks are **urgent**, P2 tasks are merely **high**.

@focus Label (+14)

The @focus label represents strategic long-term goals. A +14 bonus means a @focus task outscores a P3 medium-priority task (+4) by a significant margin, ensuring long-term goals don't get buried under routine work.

Quick-Win Bonus (+4) and Long-Task Penalty (−6)

Tasks ≤ 30 minutes get a quick-win bonus because completing small tasks builds momentum (especially for high-context workflows). Tasks ≥ 120 minutes get a penalty because they require dedicated blocks and are harder to fit into a busy day.

Section-Aware Scoring

The Kanban board structure provides additional context:

Section	Points	Rationale
Urgent/Today	+12	User explicitly placed it here
Doing	+10	Already in progress
High	+6	User-categorised as high priority
Blocked	−20	Cannot be worked on right now

Section	Points	Rationale
Rejected	−30	Discarded tasks should sink to the bottom
Backlog (high-pri)	+5	Hidden gem — high priority but forgotten
Backlog (low-pri)	−2	Deprioritise undated backlog

Backlog Penalty (−4)

Tasks with no due date **and** no @focus label get −4. This prevents the backlog from cluttering the daily briefing. If a task is important enough, the user should either set a due date or add @focus.

Tiebreaker Logic

When two tasks have the same score, they are sorted by:

1. **Due date** (earlier due date first, tasks with no date go last)
2. **Priority** (higher priority first)
3. **Alphabetical** (deterministic, stable sort)

```
scored.sort(key=lambda s: (
    -s.score,
    s.task.due_date or date.max,
    -s.task.priority,
    s.task.content.lower(),
))
```

Worked Example

Consider a task: “Fix broken kitchen faucet”

- Overdue by 2 days: $\min(2, 7) \times 20 = +40$
- Priority P2 (high): +10
- No @focus label: +0
- Duration 45 min: +0 (not quick-win, not long)
- Section “To Do - High”: +6
- Has a due date: no backlog penalty

Total: 56 points

The reasons list would be:

```
["Overdue 2d → +40", "Priority p2 → +10", "Section: High → +6"]
```

LLM Integration

The Fallback Chain

The LLM integration uses a **multi-model fallback chain** — a resilience pattern where each step is cheaper and faster than the previous one.

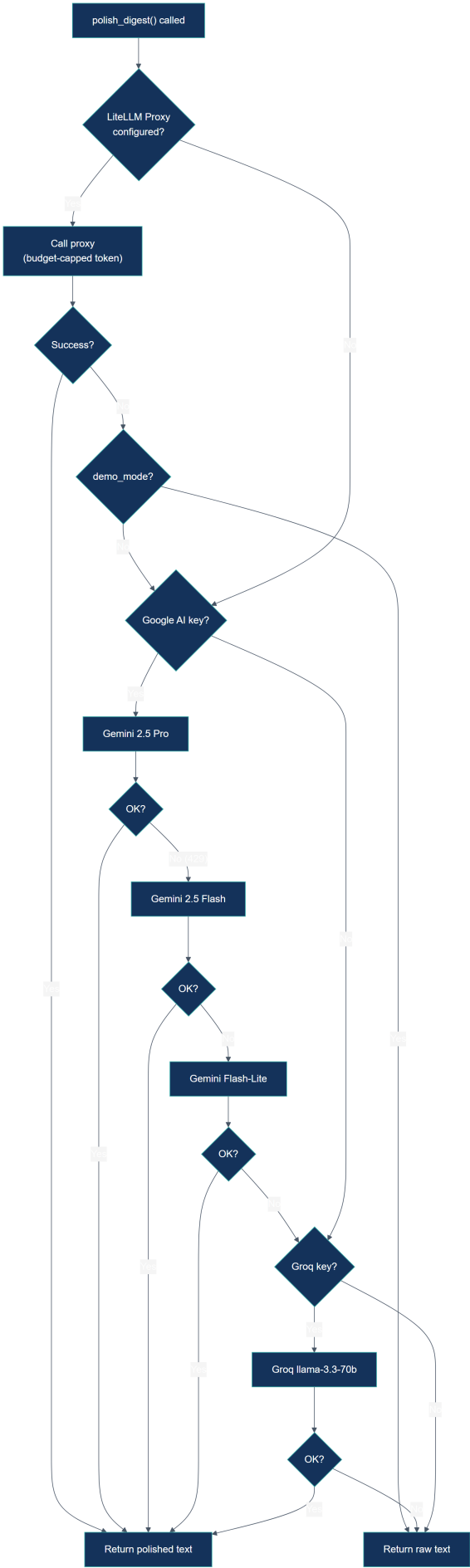


Figure 4: Diagram 3

Why not just pick the best model? Rate limits. Gemini 2.5 Pro has strict free-tier quotas. When the quota is exhausted, the API returns HTTP 429. Instead of failing, the system tries Flash (faster, higher quota), then Flash-Lite (cheapest, highest quota), then Groq (completely different provider).

The Zero-Trust Proxy

In production, all AI calls go through a self-hosted LiteLLM proxy on a VPS. The container never sees real API keys.

```
def _call_proxy(settings, text, system_prompt):
    proxy_url = settings.litellm_proxy_url.strip()

    # Security check 1: HTTPS required
    if not proxy_url.startswith("https://"):
        logger.error("Proxy URL does not use HTTPS – blocked.")
        return None

    # Security check 2: SSRF guard
    if _resolves_to_private_ip(proxy_url):
        logger.error("Proxy URL resolves to private IP – blocked.")
        return None

    url = proxy_url.rstrip("/") + "/v1/chat/completions"
    headers = {
        "Authorization": f"Bearer {settings.litellm_proxy_token}",
        "Content-Type": "application/json",
    }
    payload = {
        "model": settings.litellm_proxy_model,
        "messages": [
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": text},
        ],
        "temperature": 0.4,
        "max_tokens": 2500,
    }
    resp = requests.post(url, headers=headers, json=payload, timeout=90)
```

Two security checks before any request is made:

1. **HTTPS enforcement.** The proxy URL must use HTTPS. Without this, the proxy token would be sent in plaintext.
2. **SSRF guard.** The `_resolves_to_private_ip()` function resolves the hostname to an IP address and blocks any RFC 1918 (10.x, 172.16-31.x, 192.168.x), loopback (127.x, ::1), or link-local (169.254.x) address. This prevents an attacker who controls the config from using the proxy call to probe internal services.

SSRF Guard — Deep Dive

```
def _resolves_to_private_ip(url: str) -> bool:
    from urllib.parse import urlparse
    hostname = urlparse(url).hostname
```



```

if not hostname:
    return True # Malformed URL – block

for info in socket.getaddrinfo(hostname, None, socket.AF_UNSPEC):
    addr = ipaddress.ip_address(info[4][0])
    if addr.is_private or addr.is_loopback or \
        addr.is_link_local or addr.is_reserved:
        return True

return False # All addresses are public – allow

```

Key design decisions:

- **Fail closed.** If DNS resolution fails (`socket.gaierror`), the function returns `True` (block). An attacker cannot bypass the check by causing a DNS timeout.
- **Checks all resolved addresses.** A hostname might resolve to multiple IPs (DNS round-robin). If any of them is private, the request is blocked.
- **AF_UNSPEC** checks both IPv4 and IPv6 addresses.

Design rationale: *Why not just use a URL allowlist?*

An allowlist would work but is brittle. The proxy URL is configured once and rarely changes. SSRF protection is a defence-in-depth measure — it prevents misconfiguration from becoming a security vulnerability, regardless of the URL value.

Prompt Engineering

Each CLI command loads a specific Markdown prompt from the `prompts/` directory:

```

PROMPTS_DIR = Path(__file__).resolve().parent.parent.parent / "prompts"

def load_prompt(name: str) -> str:
    path = PROMPTS_DIR / f"{name}.md"
    return path.read_text(encoding="utf-8")

```

Example — `prompts/daily-summary.md`:

```

You are a productivity coach. Your goal is to make the
morning briefing scannable, actionable, and structured.

## Rules
- Keep ALL task names, dates, and data exactly as provided.
- Highlight the top 3 things to focus on today.
- For overdue tasks, suggest a concrete action.
- Keep total length under 500 words.

## Tone
- Professional and direct. Action-oriented.
- No fluff, no corporate speak.

```

Why separate files? Prompts are iterated independently of code. Changing a prompt does not require a code deployment. The next timer run picks up the new prompt automatically.

API Clients

Todoist Client — Cursor Pagination and Delete Protection

Cursor Pagination

Todoist API v1 uses cursor-based pagination. The client handles this transparently:

```
def _paginate(self, url, params=None):
    params = dict(params or {})
    params.setdefault("limit", 200)
    results = []
    while True:
        data = self._get(url, params=params)
        results.extend(data.get("results", []))
        cursor = data.get("next_cursor")
        if not cursor:
            break
        params["cursor"] = cursor
    return results
```

Why cursor pagination is better than offset pagination:

Feature	Offset (?page=2&limit=50)	Cursor (?cursor=abc123)
New items added between pages	Items can be missed or duplicated	Consistent — cursor marks exact position
Performance on large datasets	Slower (DB must skip N rows)	Constant time
Stateless	Yes	Yes

Delete Protection

The most distinctive feature of the Todoist client:

```
def delete_task(self, *_args, **_kwargs):
    raise NotImplementedError(
        "SAFETY: Task deletion is permanently disabled."
    )

def close_task(self, *_args, **_kwargs):
    raise NotImplementedError(
        "SAFETY: Task completion via API is permanently disabled."
    )
```

These methods exist only to raise errors. They cannot be called. The `update_task()` method also blocks destructive fields:

```
_BLOCKED_FIELDS = frozenset({"is_deleted", "checked", "in_history"})

def update_task(self, task_id, **fields):
    blocked = self._BLOCKED_FIELDS & set(fields)
    if blocked:
        raise ValueError(
            f"SAFETY: Refusing to send destructive fields: {blocked}."
        )
```

Design rationale: *Why not just use a config flag?*

A config flag can be set to the wrong value. A method that doesn't exist cannot be called by mistake. This is safety by omission — the dangerous operation is not hidden behind a flag, it's physically absent from the code.

Task Parsing

The `_parse_task()` static method converts raw Todoist JSON into a `Task` dataclass. It handles three date formats:

1. **Date-only:** "2024-06-15" → `due_date = date(2024, 6, 15)`, `due_has_time = False`
2. **With timezone:** "2024-06-15T09:00:00Z" → `due_date + due_datetime`, `due_has_time = True`
3. **Without timezone:** "2024-06-15T09:00:00" → appends +00:00 for safety

It also handles duration units:

```
if unit == "minute": duration_minutes = amount
elif unit == "hour": duration_minutes = amount * 60
elif unit == "day": duration_minutes = amount * 480 # 8-hour workday
```

Notion Client — Two-Way Sync

The Notion client handles three responsibilities:

1. **Dashboard setup** (`notion-setup`): Create the root page, Command Center, and four planner databases
2. **Forward sync** (Todoist → Notion): Upsert tasks from planner sections into Notion databases
3. **Reverse sync** (Notion → Todoist): Detect tasks created in Notion and push them back to Todoist

Text Sanitisation

Every string sent to the Notion API passes through `_sanitize_text()`:

```
_CTRL_CHARS_RE = re.compile(r"[\x00-\x08\x0b\x0c\x0e-\x1f\x7f\x80-\x9f]")

def _sanitize_text(text: str, max_len: int = 2000) -> str:
    if not isinstance(text, str):
        text = str(text)
    return _CTRL_CHARS_RE.sub("", text)[:max_len]
```

Why? The Notion API rejects blocks containing null bytes (`\x00`) or other C0 control characters. Task names copied from external sources sometimes contain invisible control characters. The regex strips them while preserving newlines and tabs.

The 2000-character limit matches Notion's maximum block length.

Block Builder Helpers

The Notion client provides 12+ helper functions for building Notion blocks:

```
_heading(level, text, emoji)    # Heading 1/2/3
_paragraph(text)                 # Plain text paragraph
_callout(text, emoji, color)    # Callout with icon
_bullet(text)                   # Bulleted list item
_numbered(text)                 # Numbered list item
_toggle(title, children)        # Collapsible section
_quote(text)                    # Block quote
_divider()                     # Horizontal rule
_code(text, language)           # Code block
_columns(*col_lists)            # Multi-column layout
_table_of_contents()            # Auto-generated TOC
_rich_text(content)             # Rich text array
```

These helpers abstract the verbose Notion block JSON format into one-line calls.

Two-Way Sync Sequence

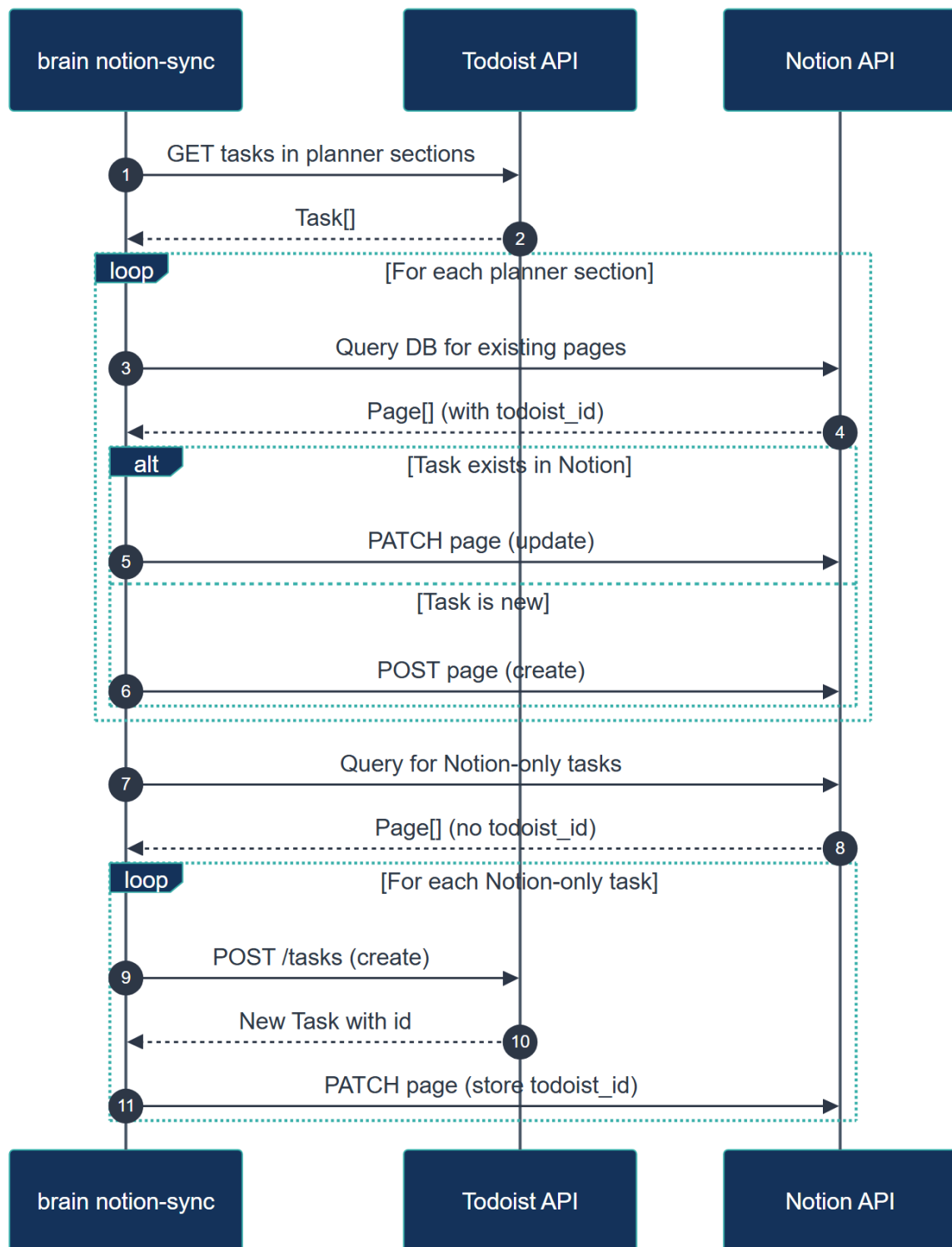


Figure 5: Diagram 4

The sync is idempotent. Running it twice produces the same result. Tasks already synced are updated, not duplicated.

Gmail — IMAP Polling

```
def get_unread_alerts(settings, hours=24):
    conn = imaplib.IMAP4_SSL(settings.gmail_imap_host, settings.gmail_imap_port)
    conn.login(settings.gmail_imap_username, settings.gmail_imap_password)
    conn.select("INBOX", readonly=True)
```

Key decisions:

- **readonly=True:** The IMAP connection never marks emails as read. The system is read-only.
- **25-alert cap:** `ids = msg_ids[0].split()[-MAX_ALERTS:]` takes only the last 25 message IDs. More would make the digest unreadably long.
- **Best-effort cleanup:** The `finally` block closes the IMAP connection with `try/except` around each operation. IMAP connections are stateful and can be in an inconsistent state if a fetch fails mid-stream.

Email Sender — STARTTLS Enforcement

```
with smtplib.SMTP(settings.smtp_host, settings.smtp_port) as server:
    server.ehlo()
    resp_code, _ = server.starttls()
    if resp_code != 220:
        logger.error("STARTTLS failed (code %d). Aborting.", resp_code)
        return False
    server.ehlo()
    server.login(settings.smtp_username, settings.smtp_password)
    server.send_message(msg)
```

Critical security detail: The `starttls()` call upgrades the connection from plaintext to TLS. The response code is explicitly checked — if STARTTLS fails, the function aborts **before** sending credentials. Without this check, credentials would be sent in plaintext over the network.

WhatsApp — CallMeBot

The simplest integration in the system:

```
# 4000 character limit
if len(message) > 3900:
    message = message[:3900] + "\n\n... (truncated)"

params = {"phone": phone, "apikey": api_key, "text": message}
requests.get(CALLMEBOT_URL, params=params, timeout=30)
```

CallMeBot is a free service that sends WhatsApp messages via HTTP GET. The 4000-character limit is enforced client-side with a clean truncation.

Board Intelligence

The Organizer Module

The `organizer.py` module provides two capabilities:

1. **Read-only analysis** (`brain analyze`): Examine the Kanban board and report on misplaced tasks, unlabeled items, duplicates, and board health.
2. **Auto-reorganization** (`brain organize`): Move tasks to their ideal sections and apply labels automatically.

Portuguese Keyword Inference

The task corpus includes Portuguese text. The organizer infers labels from task content using a keyword dictionary:

```
_LABEL_KEYWORDS = {
    "Manutenção": ["arranjar", "reparar", "pintar", "limpar", ...],
    "Tecnologia": ["proxmox", "docker", "raspberry", "vpn", ...],
    "Finanças": ["banco", "seguro", "imposto", "factura", ...],
    "Saúde": ["médico", "dentista", "farmácia", ...],
    "Lojas": ["comprar", "ikea", "amazon", "worten", ...],
    # ... more categories
}

def infer_labels(task):
    text = f"{task.content} {task.description}".lower()
    suggested = []
    for label, keywords in _LABEL_KEYWORDS.items():
        for kw in keywords:
            if kw in text:
                suggested.append(label)
                break
    return suggested
```

Design rationale: *Why hardcoded keywords instead of ML classification?*

Simplicity and transparency. There are ~100 keywords across 9 categories. The matching is instant, deterministic, and debuggable. An ML classifier would need training data, periodic retraining, and would sometimes make unexplainable mistakes. For a productivity tool with ~200 tasks, keyword matching is the right tool for this use case.

Section Placement Logic

The `suggest_section()` function determines where a task should be on the Kanban board:

```
def _ideal_section_name(task):
    if "blocked" in task.labels: return "Blocked"
    if task.is_recurring: return "Ongoing recurring"
```

```
# Don't move tasks in AI-managed planner sections
if task.section_name in ("Daily Planner", "Weekly Planner",
                        "Weekend Planner"):
    return None

# Overdue or imminent + high priority → Urgent
if task.due_date:
    days_until = (task.due_date - today).days
    if days_until <= 0:        return "To Do - Urgent/Today"
    if days_until <= 2 and task.priority >= 3:
        return "To Do - Urgent/Today"

# Priority-based placement
if task.priority == 4:        return "To Do - Urgent/Today"
if task.priority == 3:        return "To Do - High"
if task.priority == 2:        return "To Do - Normal"
if task.priority == 1:
    if task.due_date:
        return "To Do - Low"
    return "Backlog"
```

Key safeguard: Tasks in AI-managed planner sections (Daily, Weekly, Weekend) are never moved. These sections are populated by the `brain daily/weekly/weekend` commands and should not be disrupted by the organizer.

NotebookLM Integration

Why NotebookLM?

Google NotebookLM is a free AI tool that creates a searchable knowledge base from uploaded documents. By syncing Notion pages and Todoist tasks to a NotebookLM notebook, the user can ask natural-language questions like:

- “What are my overdue tasks about home maintenance?”
- “When was the last time I worked on the Proxmox setup?”
- “What did my weekly review say about project health?”

The Sync Pipeline

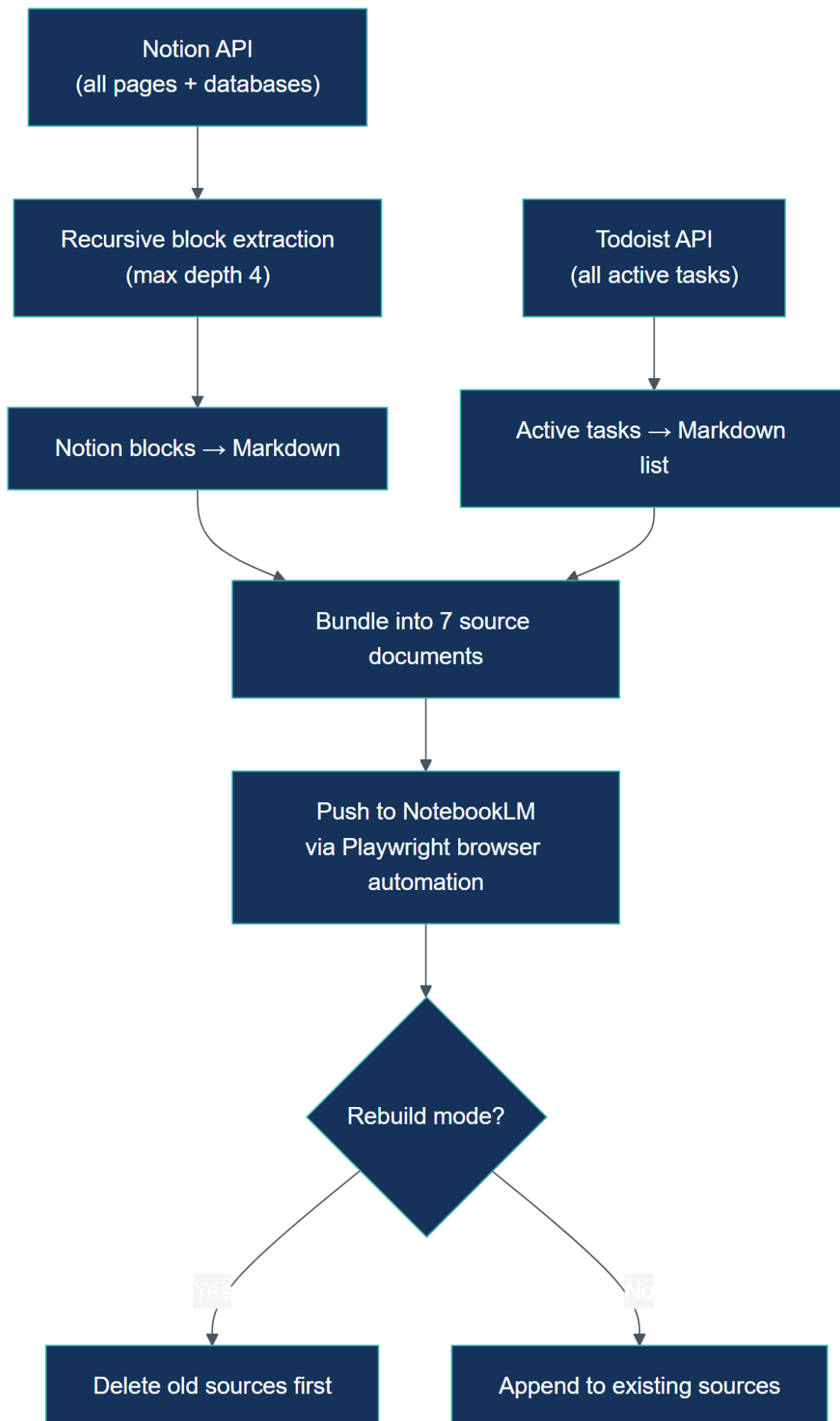


Figure 6: Diagram 5

Block-to-Markdown Conversion

The `_blocks_to_md()` function converts Notion's block JSON format into standard Markdown:

```
def _blocks_to_md(blocks):
    lines = []
    for blk in blocks:
        btype = blk.get("type", "")
        data = blk.get(btype, {})
        text = _rt(data.get("rich_text", []))

        if btype == "heading_1":    lines.append(f"# {text}")
        elif btype == "heading_2": lines.append(f"## {text}")
        elif btype == "paragraph": lines.append(text)
        elif btype == "bulleted_list_item": lines.append(f"- {text}")
        elif btype == "code":
            lang = data.get("language", "")
            lines.append(f"```{lang}\n{text}\n```")
        elif btype == "divider":   lines.append("---")
        # Unsupported types silently omitted

    return "\n".join(lines)
```

Why Playwright?

NotebookLM has no official API. The `notebooklm-py` library drives a headless Chromium browser, authenticated with exported Google session cookies. This is inherently fragile — Google can change the UI at any time — but it's the only option.

Authentication flow:

1. Run `notebooklm login` interactively on the machine
2. Authenticate with Google in the browser
3. Library saves session cookies to `~/.notebooklm/`
4. Cookies last 2–4 weeks before re-authentication is needed
5. In the LXC container, use `Xvfb + x11vnc` for headless browser access

Scheduling and Deployment

systemd Timers

Five timer/service pairs manage the schedule:

Timer	Schedule	Service Command
brain-daily.timer	Mon-Fri 07:00	brain daily
brain-sync.timer	Every 4h (06,10,14,18,22)	brain notion-sync --full
brain-weekly.timer	Sun 20:00	brain weekly
brain-weekend.timer	Fri 17:00	brain weekend
brain-notebooklm.timer	Sun 21:00	brain notebooklm-sync

Timer Configuration

```
[Timer]
OnCalendar=Mon-Fri 07:00:00
Persistent=true
Unit=brain-daily.service
```

Persistent=true is critical. If the container is offline when the timer fires (e.g., during a reboot or maintenance window), the job runs immediately on next boot. Without this, missed jobs would silently disappear.

Service Hardening

Every service unit includes kernel-level sandboxing:

```
[Service]
Type=oneshot
User=brain
Group=brain

NoNewPrivileges=true      # Process cannot gain new privileges
PrivateTmp=true           # Isolated /tmp directory
PrivateDevices=true       # No access to /dev
ProtectSystem=strict      # Root filesystem is read-only
ProtectHome=true          # /home is inaccessible
ReadWritePaths=/var/log/brain /run/brain
CapabilityBoundingSet=     # All Linux capabilities dropped
ProtectKernelTunables=true # Cannot modify /proc/sys
ProtectKernelModules=true  # Cannot load kernel modules
```

```
ProtectKernelLogs=true      # Cannot read kernel logs
ProtectControlGroups=true   # Cannot modify cgroups
RestrictNamespaces=true     # Cannot create new namespaces
RestrictSUIDSGID=true       # Cannot set SUID/SGID bits
SystemCallArchitectures=none
MemoryDenyWriteExecute=true # No writable+executable memory
```

Design rationale: *Why MemoryDenyWriteExecute?*

This prevents the process from mapping memory as both writable and executable. It blocks common exploitation techniques like JIT spraying and shellcode injection. Python doesn't need writable+executable memory for normal operation.

Docker Compose — n8n

For environments without systemd (or for users who prefer a visual scheduler), n8n runs as an optional Docker container:

```
services:
  n8n:
    image: n8nio/n8n:latest
    ports:
      - "127.0.0.1:5678:5678"
    volumes:
      - n8n_data:/home/node/.n8n
      - /opt/velaflow:/opt/velaflow:ro # Read-only mount
    environment:
      - N8N_ENCRYPTION_KEY=${N8N_ENCRYPTION_KEY:?Set in .env}
      - GROQ_API_KEY=${GROQ_API_KEY}
    healthcheck:
      test: ["CMD", "wget", "-q", "-O-", "http://localhost:5678/healthz"]
      interval: 60s
```

Key security details:

- **/opt/velaflow:ro**: The source code is mounted read-only. n8n cannot modify VelaFlow files.
- **N8N_ENCRYPTION_KEY=\${...:?Set in .env}**: Fail-fast syntax. If the encryption key is not set, Docker Compose refuses to start.
- **Health check**: The healthz endpoint is polled every 60 seconds. Docker restarts the container if it fails 3 consecutive checks.

LXC Container Provisioning

The `setup-lxc.sh` script configures the Proxmox LXC container with:

- **Unprivileged container**: Runs as a non-root user on the host
- **AppArmor profile**: generated (auto-generated restrictive profile)
- **9 capabilities dropped**: `sys_admin`, `sys_rawio`, `sys_module`, etc.
- **Docker nesting**: `nesting=1` (required for Docker inside LXC)

Security Deep Dive

The Zero-Trust Proxy Model

This is the most architecturally interesting part of VelaFlow.

The Problem with Secrets Vaults

Traditional approach: store API keys in HashiCorp Vault, AWS Secrets Manager, or a Kubernetes secret. At runtime, the application fetches the key from the vault and holds it in memory.

The problem: If the key enters the container's memory — even transiently — it can be extracted via:

- **Memory dump:** `gcore <pid>` or reading `/proc/<pid>/mem`
- **Environment inspection:** `/proc/<pid>/environ`
- **Network interception:** If TLS is misconfigured
- **Container snapshot:** Proxmox can snapshot an LXC with memory state

Anyone with root access to the Proxmox host can do any of these.

The Solution: Never Let the Key Enter

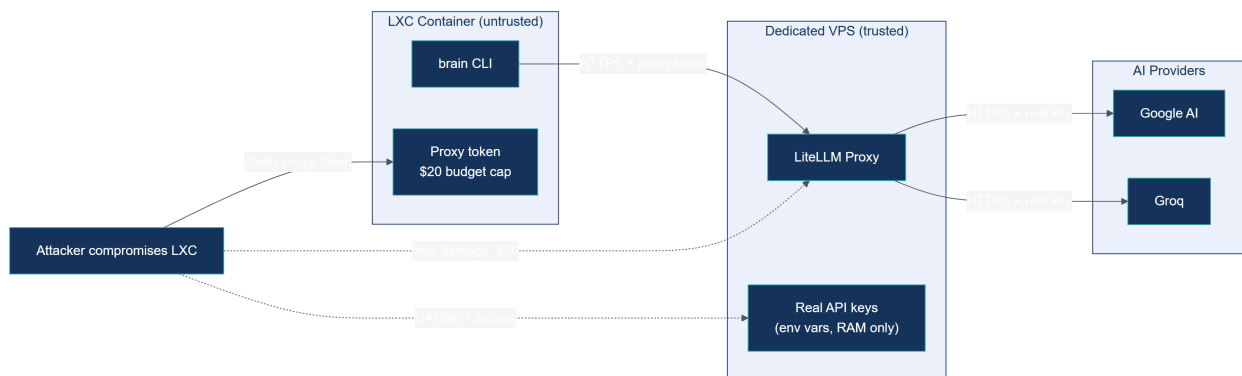


Figure 7: Diagram 6

Real API keys live exclusively on a dedicated VPS, inside a LiteLLM reverse proxy. The LXC container holds only a budget-capped proxy token. If the token is stolen:

1. The attacker can make API calls until the budget is exhausted (\$20)
2. The budget self-destructs — no more calls possible
3. Revoke the token with one curl call
4. No container redeployment needed

Demo Mode

For shared/demonstration environments, a tighter token is used:

Property	Personal token	Demo token
Budget cap	\$20/month	\$1-2 hard cap

Property	Personal token	Demo token
Expiry	Monthly reset	48 hours
IP restriction	None	Locked to container IP
Revocation	Manual	Automatic on expiry

Defence in Depth — All Six Layers

Layer	Control	Implementation
1. Network	TLS everywhere	<code>verify=True</code> on all <code>requests</code> calls. HTTPS enforced on proxy URL. STARTTLS enforced before SMTP login.
2. Process	Dedicated user	<code>brain</code> system user with no login shell, no <code>sudo</code> , no home directory.
3. Kernel	<code>systemd</code> sandboxing	<code>NoNewPrivileges</code> , <code>ProtectSystem=strict</code> , <code>MemoryDenyWriteExecute</code> , all capabilities dropped.
4. Application	Safety flags	<code>DEMO_MODE</code> disables direct key fallback. <code>BRAIN_READ_ONLY</code> blocks writes. SSRF guard on proxy URL. Delete protection on Todoist client.
5. Budget	LiteLLM caps	Per-token spend limit. Token auto-expires when budget is reached.
6. Container	LXC hardening	Unprivileged, <code>AppArmor</code> , 9 capabilities dropped.

Threat Model

Threat	Vault approach	Zero-Trust Proxy
Memory dump of container	Keys exposed	No real keys to find
Container snapshot	Keys in memory image	Proxy token only — revoke it
Network sniffing	Keys in transit (if TLS misconfigured)	Proxy token in transit — limited value
Insider with Proxmox root	Full key access	Proxy token worth \$20 max
Token theft + reuse	Full API access	Budget-capped, then self-destructs
Instant revocation	Rotate vault access key (complex)	One curl call (instant)

Testing Strategy

Test Architecture

```
tests/  
├─ test_planner.py    # 13 tests – scoring engine + digest builders  
├─ test_todoist.py    # 5 tests – task parsing + duration handling  
└─ test_llm.py        # 4 tests – fallback chain + proxy behaviour
```

Total: 276 tests, all passing.

Scoring Engine Tests (13 tests)

These are the most important tests in the codebase. They verify every scoring factor independently:

Test	What it verifies	Expected score
test_overdue_1_day	1 day overdue = +20	20
test_overdue_7_day_cap	10 days overdue is capped at 7 = +140	140
test_due_today	Due today = +25	25
test_due_tomorrow	Due tomorrow = +16	16
test_priority_p1	P1 (urgent) = +18	18
test_focus_label	@focus = +14	14
test_weekend_label_in_weekend_mode	@weekend + weekend_mode = +10	10
test_weekend_label_ignored_in_normal_mode	@weekend without weekend_mode = 0	0
test_quick_win	≤30min = +4	4
test_long_task_penalty	≥120min = −6	−6

Test	What it verifies	Expected score
<code>test_no_date_penalty</code>	No date + no @focus = -4	-4
<code>test_combined_scores</code>	Multiple factors stack correctly	Sum

Plus `test_higher_score_first` (ranking order) and `test_empty_list` (edge case).

LLM Fallback Tests (4 tests)

These tests use `unittest.mock.patch` to simulate API responses:

Test	Setup	Expected behaviour
<code>test_returns_raw_when_no_keys</code>	No API keys configured	Returns raw text unchanged
<code>test_groq_success</code>	Groq returns valid response	Returns polished text
<code>test_google_fails_falls_back_to_groq</code>	Google returns error, Groq succeeds	Returns Groq response
<code>test_both_fail_returns_raw</code>	Both APIs return errors	Returns raw text

Security Scanning Pipeline

Scanner	Scope	Result
Snyk Open Source	31 dependencies	0 vulnerabilities
Snyk Code (SAST)	54 Python files	0 issues
Bandit	Python SAST (CWE)	0 HIGH, 0 MEDIUM
pip-audit	PyPI advisories	0 known vulnerabilities

All four scanners run clean. The `# nosec` annotations in the codebase are limited to test fixtures (dummy credentials) and best-effort IMAP cleanup blocks, with explicit justification comments.

Digest Builders

Daily Briefing

The `build_daily_digest()` function produces a structured text output with these sections:

1. **Stats line:** “47 active tasks | 3 overdue | 5 due today”
2. **Top N priorities:** The highest-scored tasks with reasoning
3. **Overdue section:** Tasks past their due date (capped at 7)
4. **Due today:** Tasks due today
5. **Calendar events:** Today’s Google Calendar events
6. **Focus items:** Long-term @focus tasks
7. **Coming up:** Next 7 days
8. **Email alerts:** Unread important Gmail messages

Weekend Planner — Capacity-Aware Allocation

The weekend planner uses a **greedy allocation algorithm**:

```
capacity_min = settings.weekend_capacity_hours * 60 # per day

for scored_task in ranked:
    duration = task.duration_minutes or default_30min

    if prefer_saturday and sat_used + duration <= sat_free:
        saturday_tasks.append(task)
        sat_used += duration
    elif sun_used + duration <= sun_free:
        sunday_tasks.append(task)
        sun_used += duration
    elif sat_used + duration <= sat_free:
        saturday_tasks.append(task)
        sat_used += duration
    else:
        overflow.append(task)
```

Key features:

- **Calendar-aware:** Existing events reduce available capacity
- **@focus tasks prefer Saturday:** Morning peak energy for strategic work
- **Overflow section:** Tasks that don’t fit are explicitly listed with suggestions (“consider splitting, moving to next week, or delegating”)
- **Family time reminder:** Every weekend plan ends with a reminder that family activities take priority

Weekly Review

The `build_weekly_review()` function compares:

- Tasks completed this week (from Todoist’s completed-tasks endpoint)

- Tasks still active
- Overdue count and trend
- Project health breakdown
- Next week priorities
- Long-term goal alignment (@focus tasks)

Prompt Engineering

Prompt Design Pattern

Every prompt follows the same structure:

```
# Role definition
You are a [specific role]. Your goal is [specific goal].

## Rules
- Constraint 1 (preserve data)
- Constraint 2 (output format)
- Constraint 3 (length limit)

## Tone
- Professional and direct
- Action-oriented
- No fluff

## Output Format
[Specific structure expectations]
```

Prompt Inventory

File	Role	Key constraints
daily-summary.md	Productivity coach	Keep ALL task names exact, top 3 focus, under 500 words
weekend-planner.md	Weekend scheduler	Time-blocked Sat/Sun plan, respect capacity limits
weekly-review.md	Weekly coach	Velocity analysis, specific next-week recommendations
task-prioritization.md	Kanban consultant	Board health insights, actionable improvements

Design rationale: *Why not embed prompts in the code?*

Separation of concerns. Prompts are iterated at a different cadence than code. A prompt improvement (“add a time estimate per task”) should not require a Python code change. The Markdown files are loaded at runtime by `load_prompt()`.

Engineering Q&A

One-Line Explanations

Topic	One-liner
What is VelaFlow?	A self-hosted Python CLI that coordinates Todoist, Notion, Gmail, and Calendar into automated daily briefings with AI polish.
Why deterministic scoring?	Same input = same output. Debuggable, testable, works without AI.
Why Zero-Trust Proxy?	Real API keys never enter the container. Proxy token is budget-capped and instantly revocable.
Why systemd over cron?	Structured logging, dependency ordering, kernel-level sandboxing, persistent timers.
Why cursor pagination?	Reliable for large datasets. No shifted-window problem when data changes between pages.
Why delete protection?	Safety by omission. The method physically doesn't exist — cannot be called by accident.
Why frozen dataclass?	Immutability communicates intent. Config is read-only.
Why multi-model fallback?	Graceful degradation. Rate limits on Model A → try Model B → try Model C → raw text. Never a silent failure.
Why Playwright for NotebookLM?	No official API. Browser automation with cookie auth is the only option.
Why two-way Notion sync?	Users edit in either tool. Sync is idempotent — no duplicates, no data loss.

Architecture Decision Records

For each major decision, capture:

1. **What alternatives did you consider?**
2. **What tradeoffs did you accept?**
3. **What would you change with more time?**

Example: Zero-Trust Proxy vs. Secrets Vault

- **Alternatives:** HashiCorp Vault, AWS Secrets Manager, Kubernetes secrets
- **Tradeoff:** Requires maintaining a VPS (additional infrastructure)
- **Accepted because:** Vault still puts keys in container memory. Proxy eliminates the attack surface entirely.
- **With more time:** Add mutual TLS between container and proxy.

Example: Deterministic Scoring vs. ML Classification

- **Alternatives:** Fine-tuned classifier, RAG-based prioritisation
- **Tradeoff:** No learning from user behaviour over time
- **Accepted because:** 200 tasks don't need ML. Deterministic scoring is debuggable and reliable.
- **With more time:** Add user feedback loop to adjust weights.

Common Follow-Up Questions

Q: How do you handle API rate limits? > The LLM fallback chain automatically tries the next model. For Todoist and > Notion, the CLI exits non-zero and systemd retries on the next timer fire.

Q: What happens if the container crashes mid-sync? > The sync is idempotent. Running it again produces the same result. Partially > synced data is not corrupted — Notion pages are upserted, not inserted.

Q: How do you test without real API keys? > `unittest.mock.patch` replaces `requests.post` in test fixtures. Scoring > tests use pure dataclass inputs with no API calls.

Q: How would you scale this to multiple users? > Replace the CLI with a FastAPI service. Each user gets their own Settings > instance (loaded from a database instead of env vars). The scoring engine > and digest builders are already stateless — they take Settings as a > parameter.

Q: What's the weakest part of the system? > The NotebookLM integration. It depends on browser automation with session > cookies that expire every 2–4 weeks. Google could change the UI at any time. > Mitigation: the sync is scheduled weekly and failures are logged, not fatal.

Python Implementation Guide

This chapter is a comprehensive Python reference for anyone who wants to understand, maintain, extend, or fully rebuild VelaFlow from scratch. It covers every Python language feature used in the codebase, explains each pattern with real examples, and provides a complete step-by-step reconstruction plan. No prior VelaFlow-specific knowledge is assumed.

Why This Chapter Exists

VelaFlow is not a toy project. It is a production system with 14 modules, 31 tests, 5 API integrations, a security architecture, and automated deployment. Rebuilding it requires understanding both the Python language features and the engineering patterns that tie them together.

This chapter serves two audiences:

1. **Someone extending VelaFlow** who needs to understand how the existing code works before adding a feature.
2. **Someone building a similar system from zero** who wants a worked reference of every pattern, with copy-paste-ready templates.

Python Language Features Reference

Core Feature Map

Feature	Where used	Why
<code>@dataclass</code>	<code>Task</code> , <code>ScoredTask</code> , <code>Settings</code> , <code>CalendarEvent</code> , <code>EmailAlert</code> , <code>DigestResult</code>	Auto-generates <code>__init__</code> , <code>__repr__</code> , <code>__eq__</code> for data containers
<code>@dataclass(frozen=True)</code>	<code>Settings</code>	Makes instances immutable after creation
<code>field(default_factory=...)</code>	<code>Task.labels</code> , <code>ScoredTask.reasons</code>	Each instance gets its own mutable default (avoids shared-list bug)
<code>@property</code>	<code>CalendarEvent.duration_minutes</code>	Computed value accessed like a regular attribute
<code>@classmethod</code>	<code>Settings.from_env()</code>	Alternative constructor bound to the class, not an instance

Feature	Where used	Why
<code>@staticmethod</code>	<code>TodoistClient._parse_task()</code>	Utility function that belongs to the class namespace but needs no instance state
Type hints	Every function signature	Documents intent; enables IDE autocompletion and static analysis
Union types <code>X None</code>	<code>due_date: date None</code>	Python 3.10+ union syntax (replaces <code>Optional[X]</code>)
f-strings	Logging, digest formatting, reasons	Inline string interpolation: <code>f"Overdue {days}d"</code>
List comprehensions	Scoring, filtering, label transforms	Concise one-line data transformation
Dict comprehensions	Config loading, section mapping	Build dictionaries from iterables
Generator expressions	<code>any(kw in text for kw in keywords)</code>	Lazy evaluation, memory-efficient
Context managers (<code>with</code>)	SMTP, IMAP, file I/O	Automatic resource cleanup
<code>try/except/finally</code>	API calls, email sending, IMAP	Structured error handling with guaranteed cleanup
<code>logging</code> module	Every module	Structured, leveled output (vs. bare <code>print()</code>)
<code>pathlib.Path</code>	Config loading, prompt loading	Object-oriented path manipulation
<code>argparse</code>	<code>cli.py</code>	Command-line argument parsing with subcommands

Feature	Where used	Why
<code>unittest.mock.patch</code>	All test files	Replace real API calls with controlled fakes
<code>re</code> (regex)	Text sanitization, keyword matching	Pattern-based string operations
<code>functools.lru_cache</code>	(applicable for extension)	Memoize expensive function calls

Import Patterns

The codebase uses three import styles:

```
# 1. Standard library – top of file, alphabetical
import logging
import os
import re
from datetime import date, datetime
from pathlib import Path

# 2. Third-party – after standard library
import requests
from dotenv import load_dotenv

# 3. Lazy imports – inside functions (for optional heavy dependencies)
def get_calendar_events(settings):
    from google.oauth2.credentials import Credentials # lazy
    ...
```

When to use lazy imports: Only for optional dependencies that may not be installed (e.g., google-auth, playwright). This keeps the CLI fast even when optional features are not configured.

The `__init__.py` and `__main__.py` Pattern

```
# src/brain/__init__.py
"""VelaFlow productivity automation engine."""
__version__ = "1.0.0"

# src/brain/__main__.py
"""Allow running as: python -m brain"""
from brain.cli import main
main()
```

Why `__main__.py`? It enables `python -m brain` as an entry point. This works in any environment without installing the package. The `__init__.py` makes the directory a Python package and provides a version string.

Why this pattern over a script entry point? A script entry point (defined in `pyproject.toml` under `[project.scripts]`) requires the package to be installed via `pip install`. The `__main__.py` pattern works immediately with just `python -m brain`, which is simpler for development and container deployment.

Reading and Writing Function Signatures

Anatomy of a Function Signature

```
def score_task(
    task: Task,           # Required positional argument
    today: date,          # Required positional argument
    settings: Settings,   # Required positional argument
    weekend_mode: bool = False, # Optional keyword argument with default
) -> ScoredTask:         # Return type annotation
```

Reading this signature tells you:

1. **Name:** `score_task` — verb-noun convention, describes the action.
2. **Inputs:** `task`, `today`, `settings` are required. Calling without them raises `TypeError`.
3. **Optional:** `weekend_mode` defaults to `False`. Callers can omit it.
4. **Return:** The function returns a `ScoredTask` instance.
5. **Trailing comma:** The comma after the last parameter is a style choice that makes diffs cleaner when adding new parameters.

Type Hint Reference

Syntax	Meaning	Example
<code>str</code>	Plain string	<code>content: str</code>
<code>int</code>	Integer	<code>priority: int = 1</code>
<code>bool</code>	Boolean	<code>is_recurring: bool = False</code>
<code>date</code>	Date object (from <code>datetime</code>)	<code>due_date: date</code>
<code>datetime</code>	Date+time object	<code>due_datetime: datetime</code>
<code>X None</code>	Optional value (may be None)	<code>due_date: date None = None</code>
<code>list[str]</code>	List of strings	<code>labels: list[str]</code>
<code>dict[str, int]</code>	Dictionary mapping strings to ints	<code>points: dict[str, int]</code>
<code>tuple[int, ...]</code>	Tuple of ints (variable length)	<code>scores: tuple[int, ...]</code>
<code>Path</code>	File system path	<code>config_path: Path</code>
<code>-> None</code>	Function returns nothing	<code>def send_email(...) -> None:</code>
<code>-> bool</code>	Function returns True/False	<code>def is_valid(...) -> bool:</code>

Important: Type hints are documentation only. Python does not enforce them at runtime. They help editors (VS Code, PyCharm) provide autocompletion and catch mistakes. Tools like `mypy` can check them statically.

Dataclass Deep Dive

What `@dataclass` Does

Without `dataclass`:

```
class Task:
    def __init__(self, id, content, priority=1, labels=None):
        self.id = id
        self.content = content
        self.priority = priority
        self.labels = labels if labels is not None else []

    def __repr__(self):
        return f"Task(id={self.id!r}, content={self.content!r}, ...)"

    def __eq__(self, other):
        return (isinstance(other, Task) and
                self.id == other.id and ...)
```

With dataclass:

```
@dataclass
class Task:
    id: str
    content: str
    priority: int = 1
    labels: list[str] = field(default_factory=list)
```

The decorator auto-generates `__init__`, `__repr__`, and `__eq__`. Same behaviour, a fraction of the code.

The Mutable Default Trap

This is the most common Python dataclass bug:

```
# WRONG – all Task instances share the SAME list object
@dataclass
class Task:
    labels: list[str] = []

t1 = Task()
t2 = Task()
t1.labels.append("urgent")
print(t2.labels) # ["urgent"] – BUG! t2 was modified too
```

The fix:

```
# CORRECT – each instance gets its own list
@dataclass
class Task:
    labels: list[str] = field(default_factory=list)

t1 = Task()
t2 = Task()
t1.labels.append("urgent")
print(t2.labels) # [] – correct, independent list
```

Rule: Never use a mutable object (`list`, `dict`, `set`) as a default value. Always use `field(default_factory=...)`.

Frozen Dataclass for Configuration

```
@dataclass(frozen=True)
class Settings:
    todoist_api_token: str = ""
    smtp_host: str = "smtp.gmail.com"
    smtp_port: int = 587
    demo_mode: bool = False
```

`frozen=True` makes the instance immutable:

```
s = Settings(todoist_api_token="abc")
s.smtp_host = "other.com" # Raises FrozenInstanceError
```

Why freeze configuration?

1. Prevents accidental mutation during runtime.
2. Safe to share across threads without locks.
3. Communicates intent: “this is read-only state.”

Composition vs Inheritance

VelaFlow uses composition, not inheritance:

```
# Composition (what VelaFlow does)
@dataclass
class ScoredTask:
    task: Task          # wraps a Task
    score: int = 0
    reasons: list[str] = field(default_factory=list)

# Inheritance (what VelaFlow does NOT do)
@dataclass
class ScoredTask(Task): # inherits from Task
    score: int = 0
    reasons: list[str] = field(default_factory=list)
```

Why composition? `Task` represents a Todoist task. `ScoredTask` adds VelaFlow-specific metadata. They are different concepts. Inheritance would blur the boundary and make `ScoredTask` depend on every field of `Task`. With composition, you access the original task via `scored.task.content`.

The @property Decorator

```
@dataclass
class CalendarEvent:
    start: datetime | None = None
    end: datetime | None = None

    @property
    def duration_minutes(self) -> int:
        if self.start and self.end:
            return int((self.end - self.start).total_seconds() / 60)
        return 0
```

`@property` makes a method behave like an attribute:

```
event = CalendarEvent(start=..., end=...)
print(event.duration_minutes) # No parentheses – looks like a field
```

When to use: For values that are computed from other fields and should not be stored separately (because they would go stale).

@classmethod vs @staticmethod

```
class Settings:
    @classmethod
    def from_env(cls) -> "Settings":
        """Create Settings by reading environment variables."""
        return cls(
            todoist_api_token=os.environ.get("TODOIST_API_TOKEN", ""),
            ...
        )

class TodoistClient:
    @staticmethod
    def _parse_task(data: dict) -> Task:
        """Convert raw JSON dict to a Task dataclass."""
        return Task(
            id=data["id"],
            content=data["content"],
            ...
        )
```

Decorator	First argument	Use case
@classmethod	cls (the class itself)	Alternative constructors (from_env, from_dict)
@staticmethod	Nothing	Utility functions that belong to the class namespace

Practical difference: @classmethod knows which class called it (useful for subclassing). @staticmethod is just a regular function that lives inside the class for organizational purposes.

Control Flow Patterns

Guard Clauses (Early Return)

Instead of deeply nested if/else:

```
# Nested (hard to read)
def send_email(settings, subject, body):
    if settings.smtp_username:
        if settings.smtp_password:
            if settings.digest_to_email:
                # ... actual logic buried 3 levels deep
            ...
```

```
# Guard clauses (what VelaFlow uses)
def send_email(settings, subject, body):
    if not settings.smtp_username or not settings.smtp_password:
        return False
    if not settings.digest_to_email:
        return False
    # ... main logic at top level
    ...
```

Rule: Check preconditions first and return early. The main logic stays at the lowest indentation level.

Fail-Closed Security Checks

```
def _resolves_to_private_ip(url: str) -> bool:
    hostname = urlparse(url).hostname
    if not hostname:
        return True # BLOCK – malformed URL

    try:
        for info in socket.getaddrinfo(hostname, None, socket.AF_UNSPEC):
            addr = ipaddress.ip_address(info[4][0])
            if addr.is_private or addr.is_loopback:
                return True # BLOCK
    except socket.gaierror:
        return True # BLOCK – DNS resolution failed

    return False # ALLOW – all addresses are public
```

Pattern: The default is to block. Only the explicit success path (`return False`) allows the request through. If anything is malformed, unknown, or fails, the function blocks. This is the opposite of “fail open.”

Deterministic Sorting with Tuple Keys

```
scored.sort(key=lambda s: (
    -s.score,                # Highest score first
    s.task.due_date or date.max, # Earliest due date first (None last)
    -s.task.priority,        # Highest priority first
    s.task.content.lower(),   # Alphabetical tiebreaker
))
```

Python compares tuples element by element. If the first element is equal, it compares the second, and so on. This lets you define multi-level sort keys in a single expression.

Negation trick: `-s.score` sorts descending (highest first). For dates, you cannot negate, so tasks with no date get `date.max` (pushed to the end).

The Fallback Chain Pattern

```
def polish_digest(settings, raw_text, prompt_name):
    # Try proxy first
    if settings.litellm_proxy_url:
        result = _call_proxy(settings, raw_text, prompt)
```

```

    if result:
        return result

# Demo mode: no direct keys allowed
if settings.demo_mode:
    return raw_text

# Try Google models in order
if settings.google_ai_api_key:
    for model in [settings.google_ai_model,
                  settings.google_ai_fallback_model,
                  settings.google_ai_lite_model]:
        result = _call_google(settings, raw_text, prompt, model)
        if result:
            return result

# Try Groq
if settings.groq_api_key:
    result = _call_groq(settings, raw_text, prompt)
    if result:
        return result

# All options exhausted – return raw text
return raw_text

```

Pattern: Try each option in priority order. If one fails, try the next. If all fail, return a safe default. The system never crashes due to an unavailable AI provider.

API Client Patterns

Client Structure Template

Every API client in VelaFlow follows this structure:

```

class ExampleClient:
    """Client for the Example REST API."""

    BASE_URL = "https://api.example.com/v1"

    def __init__(self, api_token: str):
        self._session = requests.Session()
        self._session.headers["Authorization"] = f"Bearer {api_token}"
        self._session.headers["Content-Type"] = "application/json"

# — Transport layer —
    def _get(self, path: str, params: dict | None = None) -> dict:
        url = f"{self.BASE_URL}/{path.lstrip('/')}"
        resp = self._session.get(url, params=params, timeout=30)
        resp.raise_for_status()
        return resp.json()

    def _post(self, path: str, payload: dict) -> dict:
        url = f"{self.BASE_URL}/{path.lstrip('/')}"
        resp = self._session.post(url, json=payload, timeout=30)
        resp.raise_for_status()
        return resp.json()

```

```

# — Business logic —————
def get_items(self) -> list[Item]:
    raw = self._paginate("items")
    return [self._parse_item(r) for r in raw]

# — Pagination —————
def _paginate(self, path: str) -> list[dict]:
    results = []
    params = {"limit": 200}
    while True:
        data = self._get(path, params=params)
        results.extend(data.get("results", []))
        cursor = data.get("next_cursor")
        if not cursor:
            break
        params["cursor"] = cursor
    return results

# — Parsing —————
@staticmethod
def _parse_item(data: dict) -> Item:
    return Item(
        id=data["id"],
        name=data.get("name", ""),
    )

```

Delete Protection Pattern

```

def delete_item(self, *_args, **_kwargs):
    raise NotImplementedError(
        "SAFETY: Item deletion is permanently disabled."
    )

```

The method exists to prevent accidental calls. `*_args`, `**_kwargs` means it accepts any arguments (so callers get the safety error regardless of what they pass). The method cannot be “turned on” — the only way to enable deletion is to rewrite the method, which requires a deliberate code change.

Blocked Fields Pattern

```

_BLOCKED_FIELDS = frozenset({"is_deleted", "checked", "in_history"})

def update_item(self, item_id: str, **fields) -> dict:
    blocked = self._BLOCKED_FIELDS & set(fields)
    if blocked:
        raise ValueError(f"SAFETY: Refusing destructive fields: {blocked}")
    return self._post(f"items/{item_id}", fields)

```

`frozenset` is an immutable set. The `&` operator computes the intersection. If any of the caller’s fields match the blocked list, the update is refused.

Testing Patterns

Test Structure

Every test file follows the same structure:

```
import unittest
from datetime import date
from brain.models import Task, ScoredTask
from brain.planner import score_task

class TestScoring(unittest.TestCase):
    """Tests for the scoring engine."""

    def _make_task(self, **overrides) -> Task:
        """Create a Task with sensible defaults, overriding specific fields."""
        defaults = {
            "id": "test-1",
            "content": "Test task",
            "priority": 1,
        }
        defaults.update(overrides)
        return Task(**defaults)

    def test_overdue_1_day(self):
        task = self._make_task(due_date=date(2024, 6, 14))
        result = score_task(task, today=date(2024, 6, 15), settings=...)
        self.assertEqual(result.score, 20)
        self.assertIn("Overdue", result.reasons[0])
```

Key patterns:

- **_make_task helper:** Creates a Task with defaults. Tests only override the fields they care about. This keeps tests focused and avoids repeating boilerplate.
- **Descriptive test names:** `test_overdue_1_day` says exactly what is being tested.
- **Assert both value and reason:** Check the score AND that the reasons list contains the expected explanation.

Mocking External APIs

```
from unittest.mock import patch, MagicMock

class TestLLM(unittest.TestCase):

    @patch("brain.llm.requests.post")
    def test_groq_success(self, mock_post):
        # Arrange: configure the mock to return a fake response
        mock_resp = MagicMock()
        mock_resp.status_code = 200
        mock_resp.json.return_value = {
            "choices": [{"message": {"content": "Polished text"}}]
        }
        mock_post.return_value = mock_resp

        # Act
        result = polish_digest(settings, "raw text", "daily-summary")

        # Assert
```

```
self.assertEqual(result, "Polished text")
mock_post.assert_called_once()
```

What `@patch` does: Temporarily replaces `requests.post` with a `MagicMock` object. The test controls what the mock returns, so no real HTTP call is made. After the test, the original function is restored automatically.

Running Tests

```
# Run all tests
python -m pytest tests/ -v

# Run a specific test file
python -m pytest tests/test_planner.py -v

# Run a single test
python -m pytest tests/test_planner.py::TestScoring::test_overdue_1_day -v

# Run with coverage
python -m pytest tests/ --cov=brain --cov-report=term-missing
```

Error Handling Patterns

The `try/except/finally` Pattern

```
def get_unread_alerts(settings, hours=24):
    conn = None
    try:
        conn = imaplib.IMAP4_SSL(settings.gmail_imap_host)
        conn.login(settings.gmail_imap_username, settings.gmail_imap_password)
        conn.select("INBOX", readonly=True)
        # ... fetch messages ...
        return alerts
    except imaplib.IMAP4.error as exc:
        logger.error("IMAP error: %s", exc)
        return []
    finally:
        if conn:
            try:
                conn.close()
            except Exception:
                pass
            try:
                conn.logout()
            except Exception:
                pass
```

Why `finally`? The IMAP connection must be closed regardless of whether the fetch succeeded or failed. The inner `try/except` blocks in `finally` handle the case where the connection is already broken.

Logging vs Raising

Situation	Action
Recoverable failure (API timeout, model unavailable)	<code>logger.warning(...)</code> + try next option
Configuration error (missing required key)	<code>logger.error(...)</code> + return safe default
Programming error (impossible state)	<code>raise ValueError(...)</code>
Security violation (blocked fields, SSRF)	raise or return True (block)

Rule: Only raise exceptions for conditions the caller must handle. For conditions where the system can degrade gracefully, log and continue.

Package Structure

Project Layout

```
VelaFlow/
  pyproject.toml      # Package metadata (PEP 621)
  src/
    brain/            # Main package
      __init__.py      # Package marker + version
      __main__.py      # python -m brain entry point
      cli.py           # Argparse CLI (top-level orchestrator)
      config.py        # Settings dataclass + env loading
      models.py        # Data models (Task, ScoredTask, etc.)
      planner.py       # Scoring engine + digest builders
      todoist.py       # Todoist API client
      notion.py        # Notion API client
      llm.py           # LLM integration + SSRF guard
      organizer.py     # Board intelligence
      email_sender.py  # SMTP delivery
      gmail.py         # IMAP polling
      whatsapp.py      # CallMeBot delivery
      calendar_ctx.py  # Google Calendar
      notebooklm.py    # NotebookLM sync
    tests/
      __init__.py
      test_planner.py  # Scoring + digest tests
      test_todoist.py  # Task parsing tests
      test_llm.py      # Fallback chain tests
    config/
      .env.example     # Template configuration
    prompts/
      daily-summary.md # LLM prompts (loaded at runtime)
    scripts/
      install.sh        # LXC provisioning
      brain-daily.service # systemd units
```

pyproject.toml Explained

```
[project]
name = "brain"
```

```

version = "1.0.0"
requires-python = ">=3.11"

dependencies = [
    "requests>=2.31",      # HTTP client for all APIs
    "python-dotenv>=1.0",  # Load .env files
]

[project.optional-dependencies]
calendar = ["google-auth", "google-auth-oauthlib", "google-api-python-client"]
notebooklm = ["notebooklm-py[browser]", "playwright"]
dev = ["pytest", "pytest-cov", "bandit", "pip-audit"]

[project.scripts]
brain = "brain.cli:main" # Installs `brain` command when pip-installed

```

Why optional dependencies? Not every deployment needs Google Calendar or NotebookLM. Keeping them optional means the core system installs with just `requests` and `python-dotenv`. Install extras with: `pip install -e ".[calendar,notebooklm,dev]"`.

Concurrency and I/O Patterns

Sequential by Design

VelaFlow is deliberately single-threaded and synchronous. Each CLI command runs one pipeline: ingest, process, polish, deliver. There is no need for concurrency because:

1. The system runs on a timer (once per day at 07:00).
2. Each API call takes 1-5 seconds. Total pipeline: 15-30 seconds.
3. Adding concurrency would complicate error handling significantly.

Timeout Discipline

Every external call has an explicit timeout:

```

requests.post(url, json=payload, timeout=90)    # LLM calls (slow)
requests.get(url, params=params, timeout=30)    # API calls (fast)

```

Without a timeout, a hung connection would block the entire pipeline indefinitely. With systemd's `TimeoutStartSec=300`, the service is killed after 5 minutes regardless.

Security Implementation Patterns

HTTPS Enforcement

```

if not proxy_url.startswith("https://"):
    logger.error("Proxy URL does not use HTTPS – blocked.")
    return None

```

STARTTLS Verification

```

resp_code, _ = server.starttls()
if resp_code != 220:
    logger.error("STARTTLS failed (code %d). Aborting.", resp_code)
    return False

```

Input Sanitisation

```

_CTRL_CHARS_RE = re.compile(r"[\x00-\x08\x0b\x0c\x0e-\x1f\x7f\x80-\x9f]")

def _sanitize_text(text: str, max_len: int = 2000) -> str:
    return _CTRL_CHARS_RE.sub("", str(text))[:max_len]

```

Every string sent to external APIs passes through sanitisation. Control characters are stripped. Length is capped. This prevents injection attacks and API rejections.

Complete Step-by-Step Rebuild Plan

This section provides a detailed reconstruction plan for building the entire system from zero. Each phase is independent and testable.

Phase 1: Data Models and Configuration (Day 1)

Create `models.py` with these exact dataclasses:

```

from dataclasses import dataclass, field
from datetime import date, datetime

@dataclass
class Task:
    id: str
    content: str
    description: str = ""
    project_id: str = ""
    project_name: str = ""
    section_id: str = ""
    section_name: str = ""
    priority: int = 1
    labels: list[str] = field(default_factory=list)
    due_date: date | None = None
    due_datetime: datetime | None = None
    due_has_time: bool = False
    duration_minutes: int | None = None
    url: str = ""
    is_recurring: bool = False
    added_at: datetime | None = None
    parent_id: str | None = None

@dataclass
class ScoredTask:
    task: Task
    score: int = 0
    reasons: list[str] = field(default_factory=list)

@dataclass

```

```

class CalendarEvent:
    summary: str
    start: datetime | None = None
    end: datetime | None = None
    all_day: bool = False

    @property
    def duration_minutes(self) -> int:
        if self.start and self.end:
            return int((self.end - self.start).total_seconds() / 60)
        return 0

@dataclass
class EmailAlert:
    subject: str
    sender: str
    sent_at: datetime | None = None

@dataclass
class DigestResult:
    subject: str
    body_text: str
    body_html: str = ""

```

Create `config.py` with a frozen Settings dataclass that reads from `os.environ`. Use `python-dotenv` to load `config/.env` at startup.

Test: Instantiate every dataclass with sample data. Verify frozen settings cannot be mutated.

Phase 2: Todoist Client (Day 2-3)

Build `todoist.py` with these methods:

1. `__init__(api_token)` — create a `requests.Session` with Bearer auth.
2. `_get(path, params)` — HTTP GET with timeout.
3. `_paginate(path)` — cursor-based pagination loop.
4. `get_tasks()` — fetch all active tasks.
5. `get_projects()` — fetch all projects.
6. `get_sections(project_id)` — fetch sections for a project.
7. `_parse_task(data)` — convert raw JSON to Task dataclass.
8. `delete_task()` — raise `NotImplementedError`.
9. `close_task()` — raise `NotImplementedError`.
10. `update_task(task_id, **fields)` — with blocked fields check.

Test: Write `test_todoist.py` with mock responses. Verify date parsing, duration parsing, priority mapping, and delete protection.

Phase 3: Scoring Engine (Day 4-5)

Build `planner.py` with:

1. `score_task(task, today, settings, weekend_mode)` — apply all scoring factors and return `ScoredTask`.
2. `rank_tasks(tasks, today, settings, weekend_mode)` — score and sort.
3. `build_daily_digest(settings)` — full daily pipeline.
4. `build_weekend_digest(settings)` — capacity-aware weekend planner.

5. `build_weekly_review(settings)` — weekly analysis.
6. `load_prompt(name)` — read prompt from `prompts/` directory.

Test: Write `test_planner.py` with one test per scoring factor. This is the most critical test file — the scoring engine is the heart of the system.

Phase 4: Delivery Adapters (Day 6)

Build `email_sender.py` (SMTP + STARTTLS), `whatsapp.py` (CallMeBot), and `gmail.py` (IMAP polling). Each module is independent and small (50-100 lines).

Test: Mock `smtplib.SMTP` and `imaplib.IMAP4_SSL` to verify the connection flow without real credentials.

Phase 5: LLM Integration (Day 7-8)

Build `llm.py` with the fallback chain: proxy, Google models, Groq, raw text. Implement the SSRF guard as a separate function.

Test: Write `test_llm.py` with mocked `requests.post`. Verify fallback order and raw-text default.

Phase 6: Notion Sync (Day 9-12)

Build `notion.py` — the largest module. Start with read operations (query databases), then add write operations (upsert pages), then two-way sync.

Test: Mock the Notion API responses. Verify idempotent upsert behaviour.

Phase 7: Board Intelligence (Day 13-14)

Build `organizer.py` with keyword inference, section placement logic, and board analysis reporting.

Phase 8: CLI and Integration (Day 15-16)

Build `cli.py` with `argparse` subcommands that wire everything together. Add `systemd` units and the deployment script.

Adding a New Feature: Checklist

When extending VelaFlow with a new integration, follow this checklist:

1. **Add configuration fields** to `Settings` in `config.py` with sensible defaults.
2. **Add the corresponding environment variables** to `config/.env.example`.
3. **Create the API client module** following the template in section 16.6.1.
4. **Add parsing tests** with mock responses.
5. **Wire into the CLI** by adding a new subcommand in `cli.py`.
6. **Add a systemd timer** if the feature should run on a schedule.
7. **Update the documentation** (this book, the architecture docs, README).
8. **Run the full test suite** and security scanners before merging.

Common Pitfalls and Debugging

Symptom	Likely cause	Fix
<code>TypeError: missing required argument</code>	Forgot a required parameter	Check the function signature
<code>FrozenInstanceError</code>	Trying to modify a frozen dataclass	Create a new instance instead
All tasks share the same labels	Mutable default in dataclass	Use <code>field(default_factory=list)</code>
<code>requests.exceptions.Timeout</code>	API server slow or unreachable	Check network; the fallback chain handles this
<code>KeyError</code> in JSON parsing	API response format changed	Update the <code>_parse_*</code> method
Score seems wrong	Check the reasons list	<code>print(scored_task.reasons)</code> shows every point
<code>NotImplementedError: SAFETY</code>	Tried to call a blocked method	This is intentional — the operation is disabled
STARTTLS failed	SMTP server rejected TLS upgrade	Verify SMTP host and port (587 for STARTTLS)
<code>UnicodeDecodeError</code>	File not UTF-8	Use <code>encoding="utf-8"</code> in all <code>open()</code> calls

Recommended Practice Loop

For fast skill recovery, use this iterative loop while reading the codebase:

1. **Pick one function** (start with `score_task` in `planner.py`).
2. **Read the function** and write down what each line does.
3. **Rewrite it** in a scratch file from memory (no copy-paste).
4. **Run the tests** for that function: `pytest tests/test_planner.py -v -k score`.
5. **Compare** your version with the original. Note differences.
6. **Add one small improvement** (better variable name, extra edge case).
7. **Run tests again** to verify the improvement works.
8. **Document** what you learned in one paragraph.

Repeat with one function per session. Suggested order:

1. `score_task` (`planner.py`) — pure function, easy to test
2. `_parse_task` (`todoist.py`) — JSON parsing, date handling
3. `send_digest_email` (`email_sender.py`) — network I/O, security checks
4. `polish_digest` (`llm.py`) — fallback chain pattern
5. `_paginate` (`todoist.py`) — cursor pagination loop
6. `infer_labels` (`organizer.py`) — keyword matching
7. `_blocks_to_md` (`notebooklm.py`) — data format conversion

After completing all seven, you will have hands-on experience with every major pattern in the codebase.

Complete File Inventory

Path	Lines	Purpose
src/brain/__init__.py	~5	Package init, version string
src/brain/__main__.py	~5	<code>python -m brain</code> entry point
src/brain/cli.py	~500	Argparse CLI, 10 subcommands
src/brain/config.py	~200	Frozen Settings dataclass, env loading
src/brain/models.py	~70	Task, ScoredTask, CalendarEvent, EmailAlert, DigestResult
src/brain/todoist.py	~310	Todoist API v1 client, cursor pagination, delete protection
src/brain/notion.py	~800	Notion dashboard + two-way sync + block builders
src/brain/planner.py	~500	Scoring engine, 4 digest builders, prompt loading
src/brain/llm.py	~300	Multi-model fallback chain, SSRF guard
src/brain/organizer.py	~400	Board analysis, Portuguese keyword inference
src/brain/email_sender.py	~60	SMTP + STARTTLS
src/brain/gmail.py	~100	IMAP4_SSL polling
src/brain/whatsapp.py	~70	CallMeBot integration
src/brain/calendar_ctx.py	~100	Google Calendar OAuth2
src/brain/notebooklm.py	~300	Notion → NotebookLM sync
tests/test_planner.py	~300	13 scoring tests + digest tests
tests/test_todoist.py	~100	5 task parsing tests
tests/test_llm.py	~100	4 fallback chain tests
scripts/brain-daily.service	~30	systemd service with sandboxing
scripts/brain-daily.timer	~10	Mon-Fri 07:00 schedule
scripts/install.sh	~200	LXC provisioning script
scripts/setup-lxc.sh	~100	Proxmox host-side LXC setup
scripts/setup-litellm-proxy.sh	~50	VPS proxy configuration
prompts/daily-summary.md	~25	Daily briefing system prompt

Path	Lines	Purpose
<code>prompts/weekend-planner.md</code>	~30	Weekend planner prompt
<code>prompts/weekly-review.md</code>	~30	Weekly review prompt
<code>prompts/task-prioritization.md</code>	~30	Board analysis prompt
<code>docker-compose.yml</code>	~30	n8n container definition
<code>pyproject.toml</code>	~60	PEP 621 packaging

Glossary

Term	Definition
LXC	Linux Containers — OS-level virtualisation. Lighter than VMs, heavier than Docker.
Proxmox	Open-source virtualisation platform. Manages VMs and LXC containers.
systemd	Linux init system and service manager. Handles scheduling, logging, and process isolation.
LiteLLM	Open-source proxy that provides a unified OpenAI-compatible API for multiple LLM providers.
SSRF	Server-Side Request Forgery — an attack where the server is tricked into making requests to internal services.
STARTTLS	Protocol extension that upgrades a plaintext connection to TLS. Used by SMTP.
IMAP	Internet Message Access Protocol — used to read email from a server.
Cursor pagination	Pagination method that uses an opaque token to mark position, rather than page numbers.
Frozen dataclass	Python dataclass with <code>frozen=True</code> — instances are immutable after creation.
Idempotent	An operation that produces the same result regardless of how many times it's executed.
Playwright	Browser automation library. Used here to drive headless Chromium for NotebookLM.
AppArmor	Linux security module that restricts programs' capabilities using profiles.
CallMeBot	Free web service that sends WhatsApp messages via HTTP API.
PEP 621	Python standard for declaring project metadata in <code>pyproject.toml</code> .

End of document.